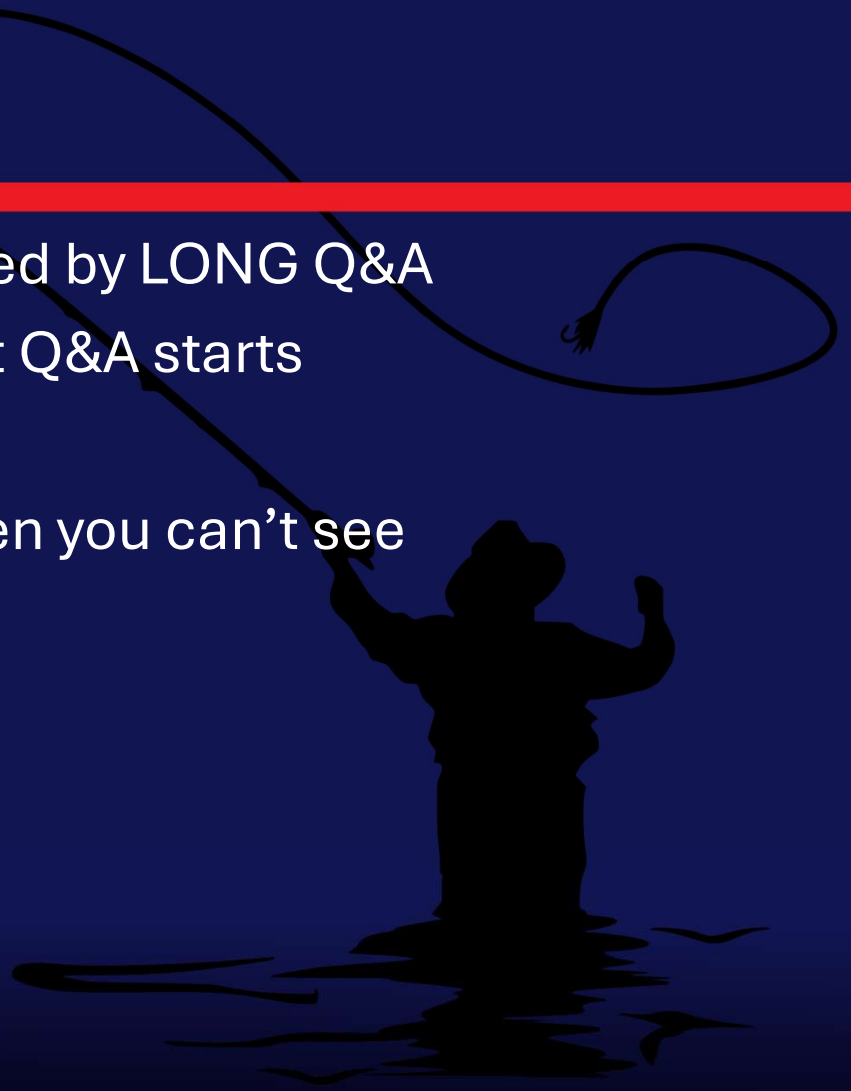
Integrating Azure Monitor Alerts with ITSM Systems

Integrating Azure Monitor seamlessly with your business processes

Attention

- MMS sessions are 60-75 minutes followed by LONG Q&A
- Please hold detailed questions until that Q&A starts
- Feel free to ask clarification questions
- Remind the speakers to use Zoom It when you can’t see



Speakers

John Joyner

Senior Director, Technology @ Corsica Technologies

john.joyner@corsicatech.com

in/johnjoyner

Brian Wren

Principal Content Developer @ Microsoft

bwren@microsoft.com

in/brianwrenlb

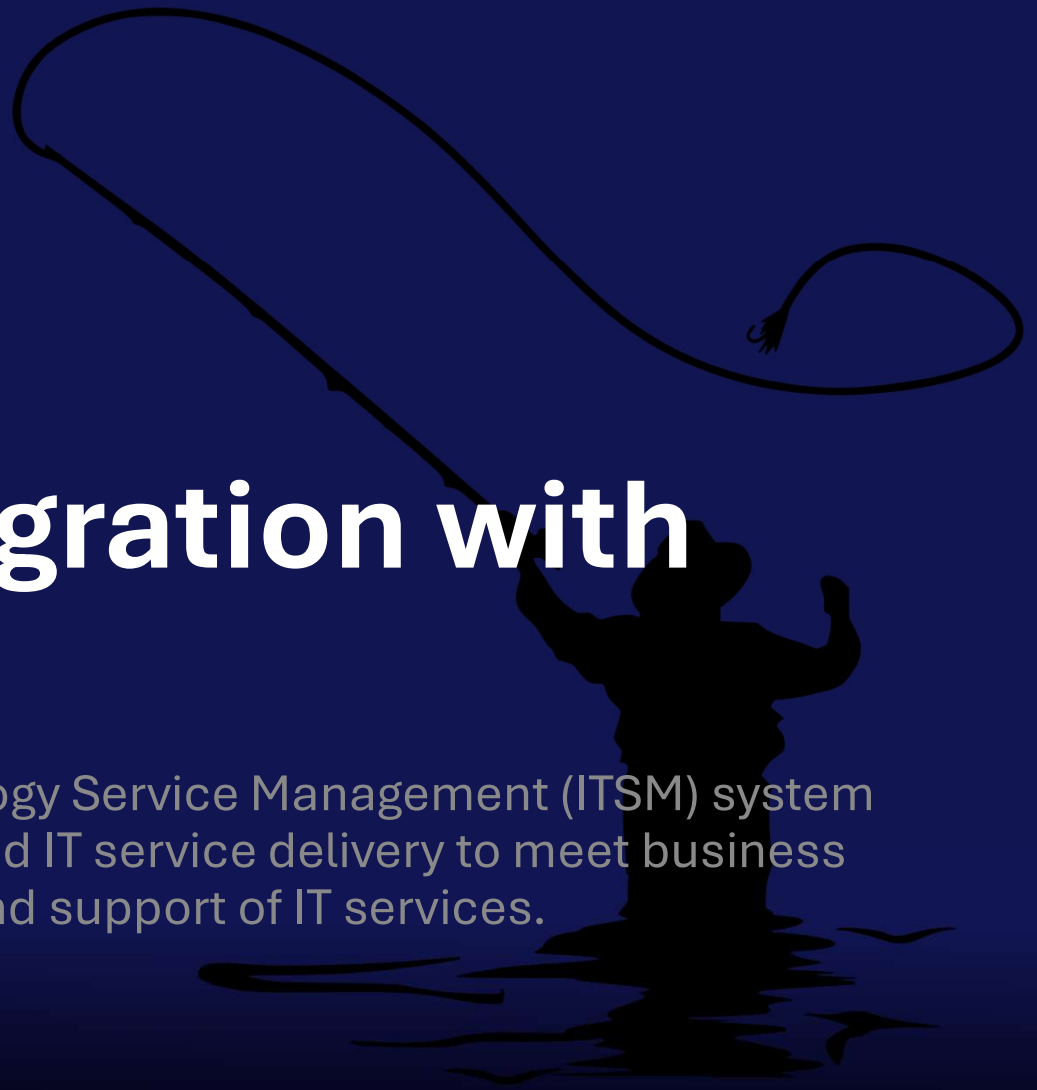
Can’t see our slides? Can’t hear? Need to repeat the question? Call us out!

What you will learn

- Utilize Azure Monitor alert control points for effective bi-directional sync with your ITSM system.
- Modify existing GitHub Logic Apps to integrate with ServiceNow, Zendesk, JIRA, Freshdesk, ManageEngine, Autotask, BMC, and Ivanti.
- Implement best practices for storing ITSM API secrets securely in Azure Key Vault, ensuring safe input and output management.
- Access tools in GitHub and learn how they fit into a comprehensive 14-step project plan to confidently achieve your integration objectives.

Monitoring integration with ITSM

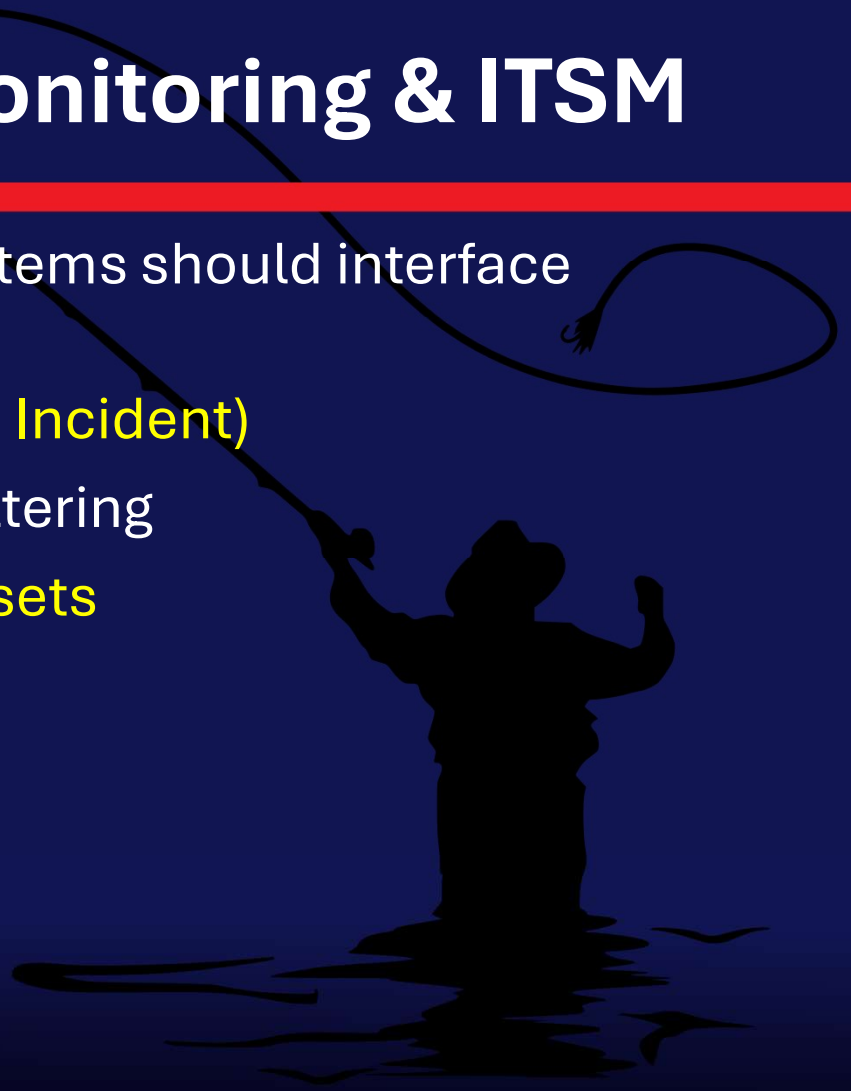
Having an effective Information Technology Service Management (ITSM) system is required for management of end-to-end IT service delivery to meet business goals, including the creation, delivery, and support of IT services.



What ITIL 4 says about Monitoring & ITSM

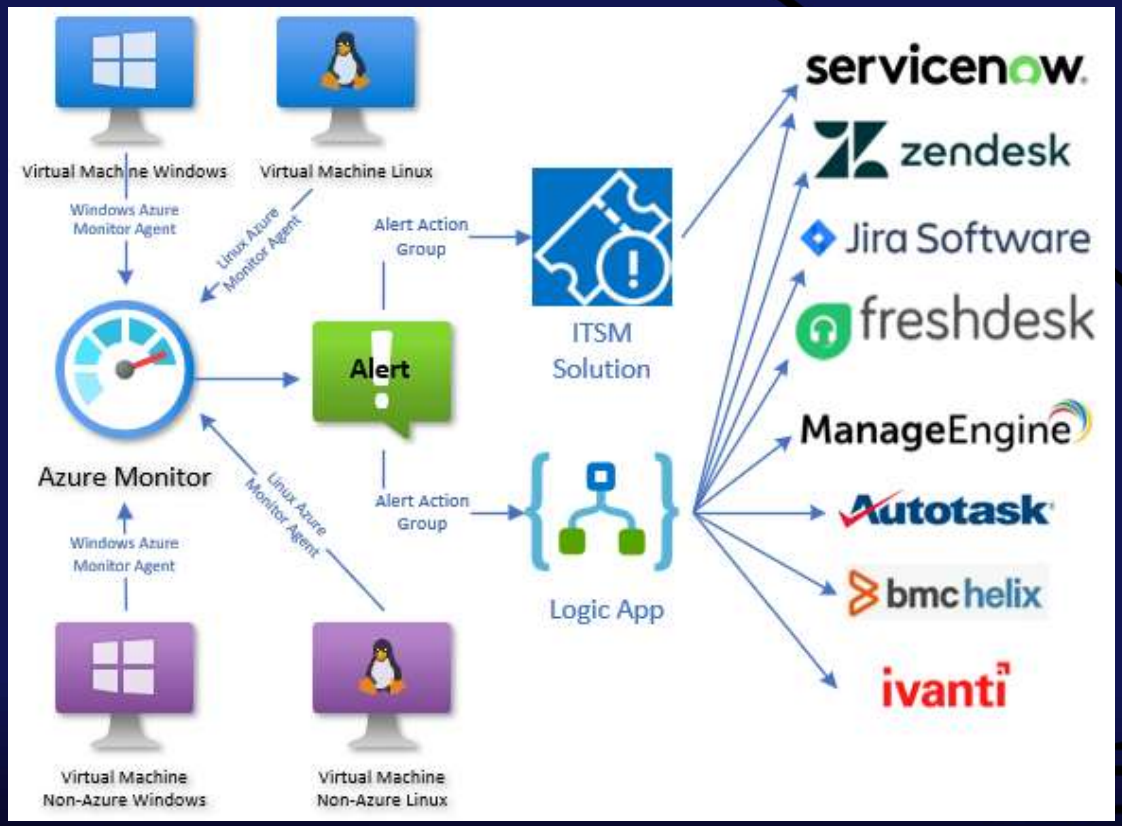
Key ITIL guidelines for how monitoring systems should interface with ITSM include:

1. Automated Incident Creation (Event to Incident)
2. Intelligent Event Categorization and Filtering
3. Integration with CMDB and Service Assets
4. Directing to Proper Workflows
5. Single Pane of Glass Visibility



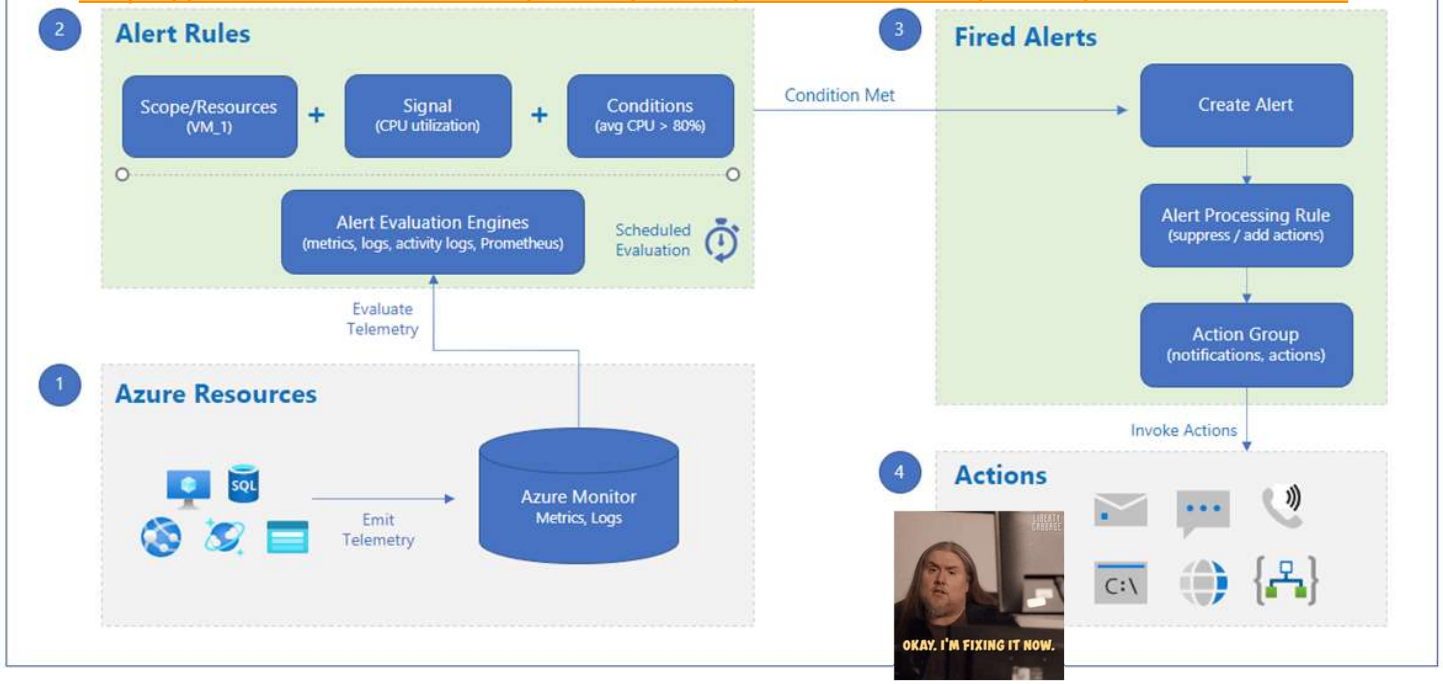
MMS

Azure Monitor & ITSM systems



What are Azure Monitor Alerts?

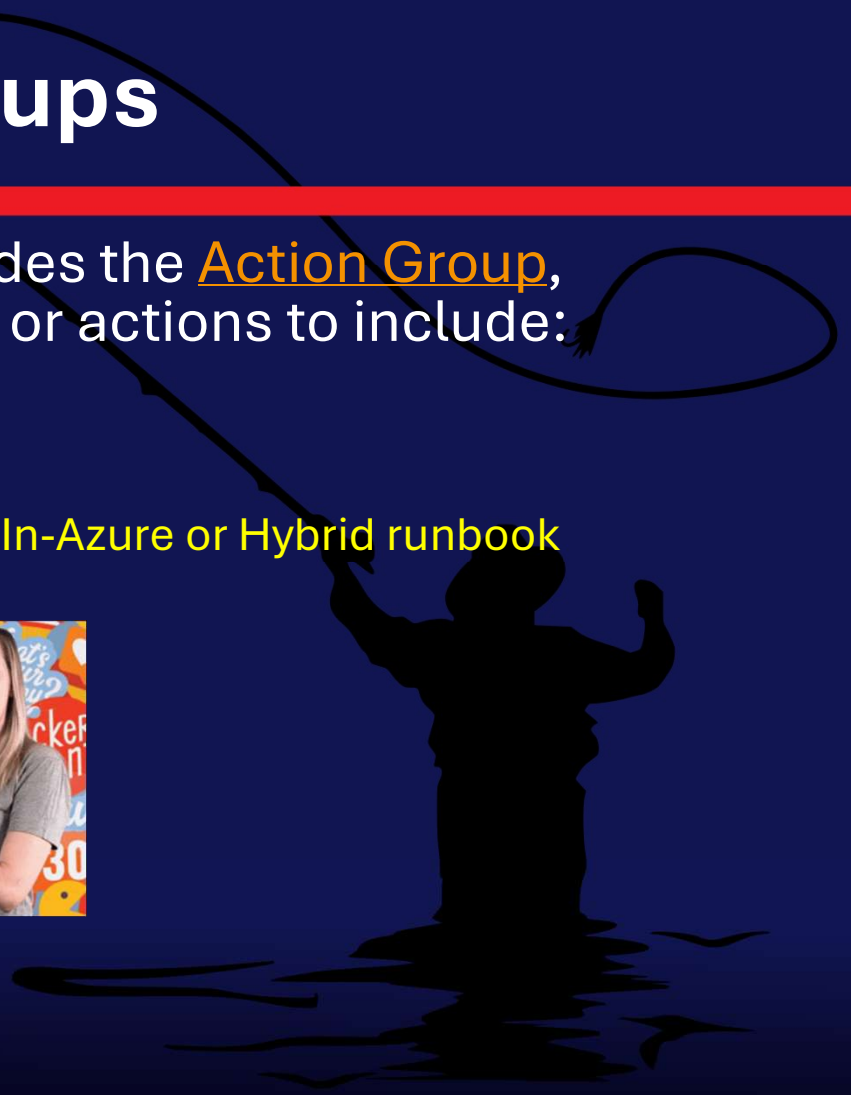
https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/alerts-overview



Azure Monitor Action Groups

Azure Monitor alerting configuration includes the Action Group, which is a list of one or more notifications or actions to include:

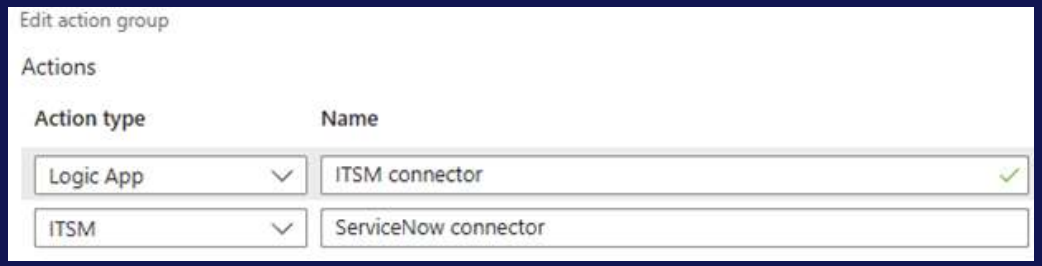
- Email Azure Resource Manager Role
- Email/SMS message/Push/Voice
- Azure Automation Runbook (PowerShell or Python, In-Azure or Hybrid runbook worker)
- Azure Function
- Event Hub
- ITSM
- Logic app
- Secure Webhook
- Webhook

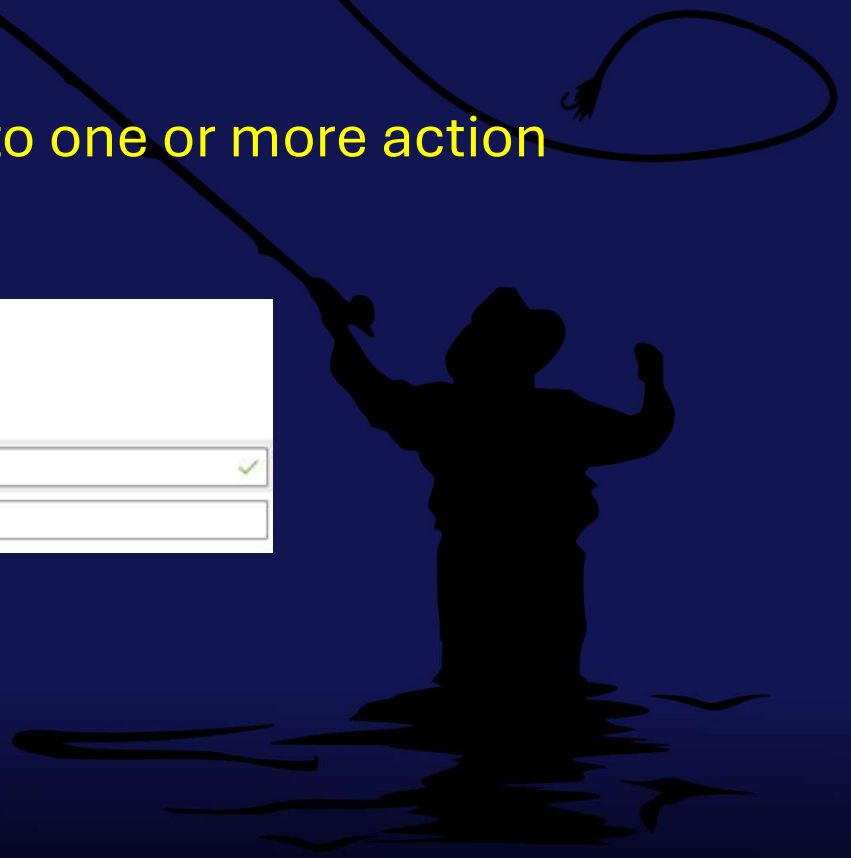


MMS

Selecting Logic app and ITSM actions in an Azure Monitor action group configuration

Each Azure Monitor alert rule is mapped to one or more action groups.

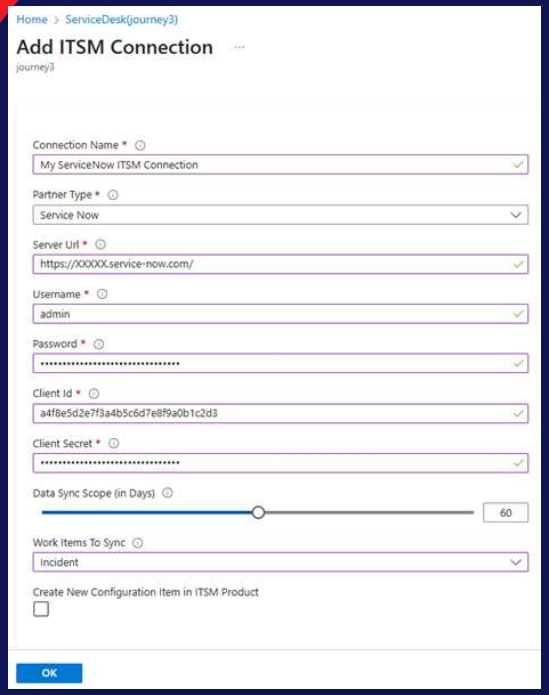


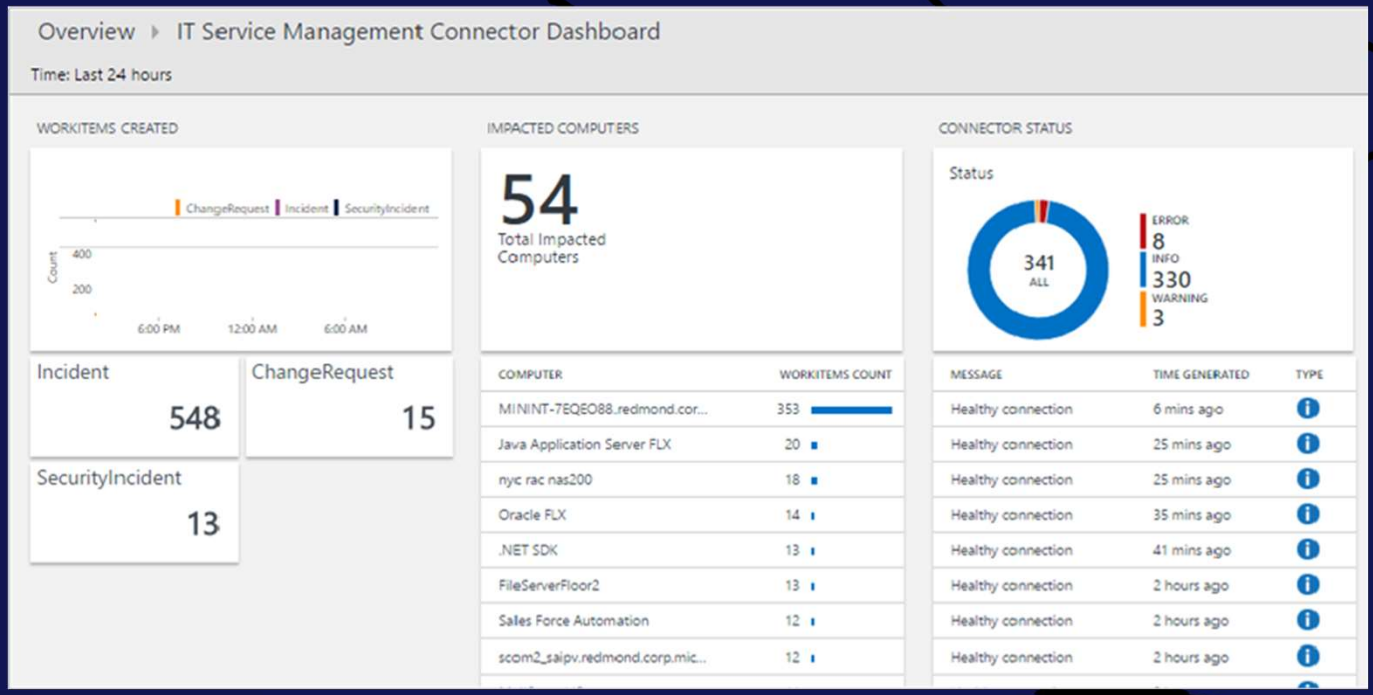


ITSM Action Group action

- Azure Monitor action groups include the ITSM action type, which in the past, supported several different ITSM tools including Cherwell, Provance, and BMC Helix.
- However, currently only ServiceNow is supported for new ITSM connections, and the entire ITSM connector type is essentially deprecated.
- The Azure Monitor ITSM connector for creating ServiceNow alerts is being retired. The deprecation process began in September 2022 and is scheduled for full retirement on September 30, 2025.
- While the ITSM connector creation process is still present in the Azure portal (May 2026), at this point the Azure Monitor ITSM connector for ServiceNow should only be created for concept validation.

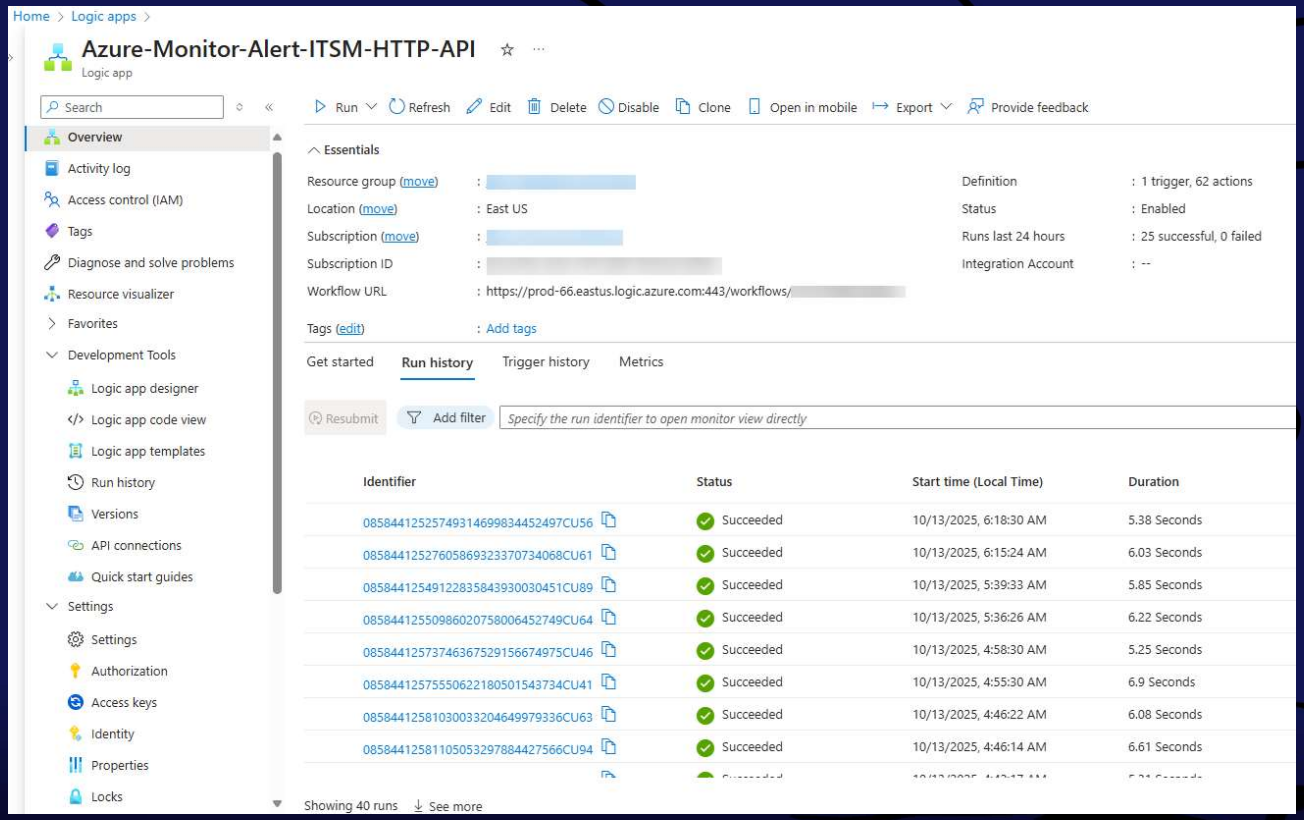
ITSM Action Group action: ServiceNow

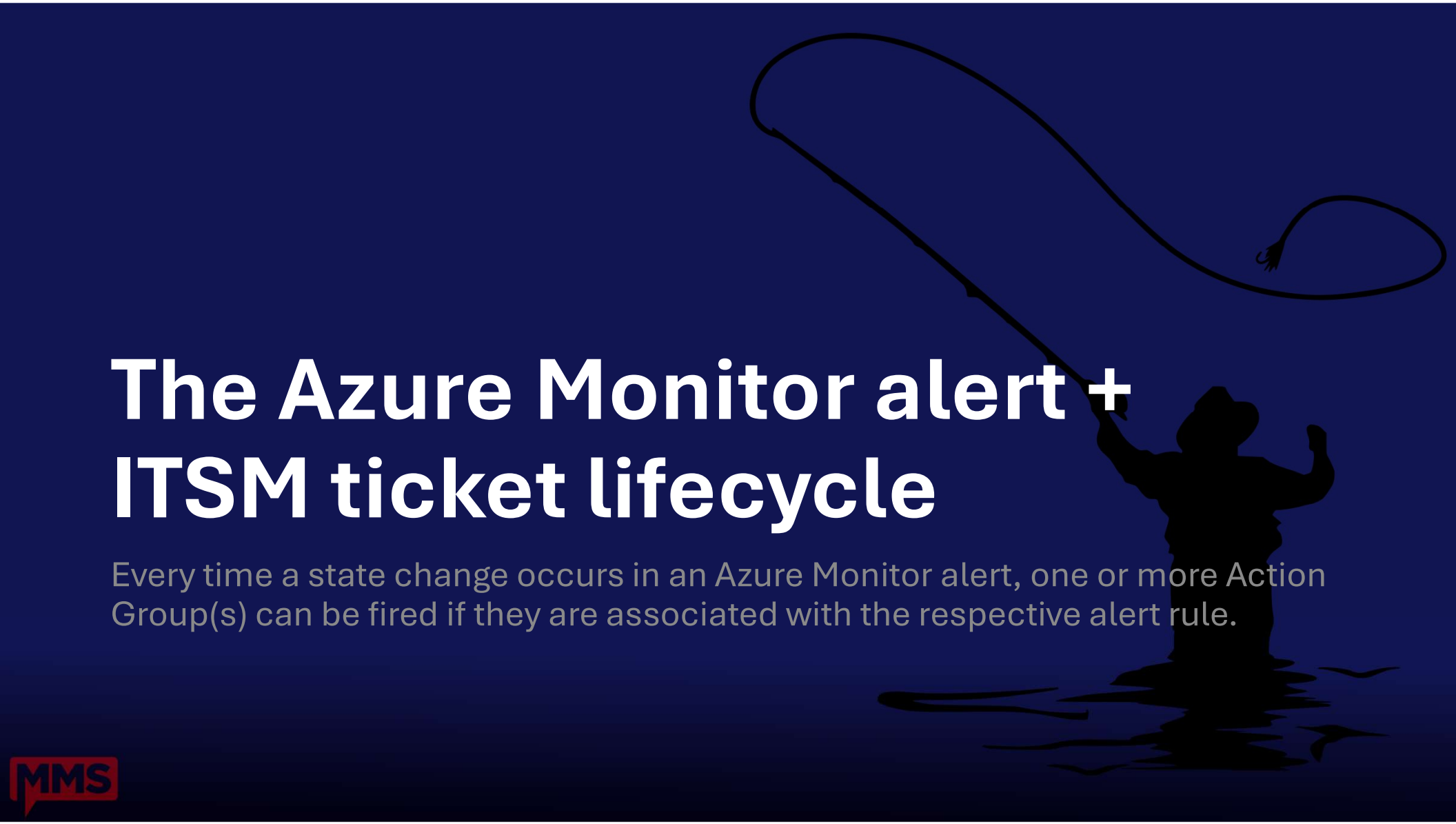




Logic App Action Group action

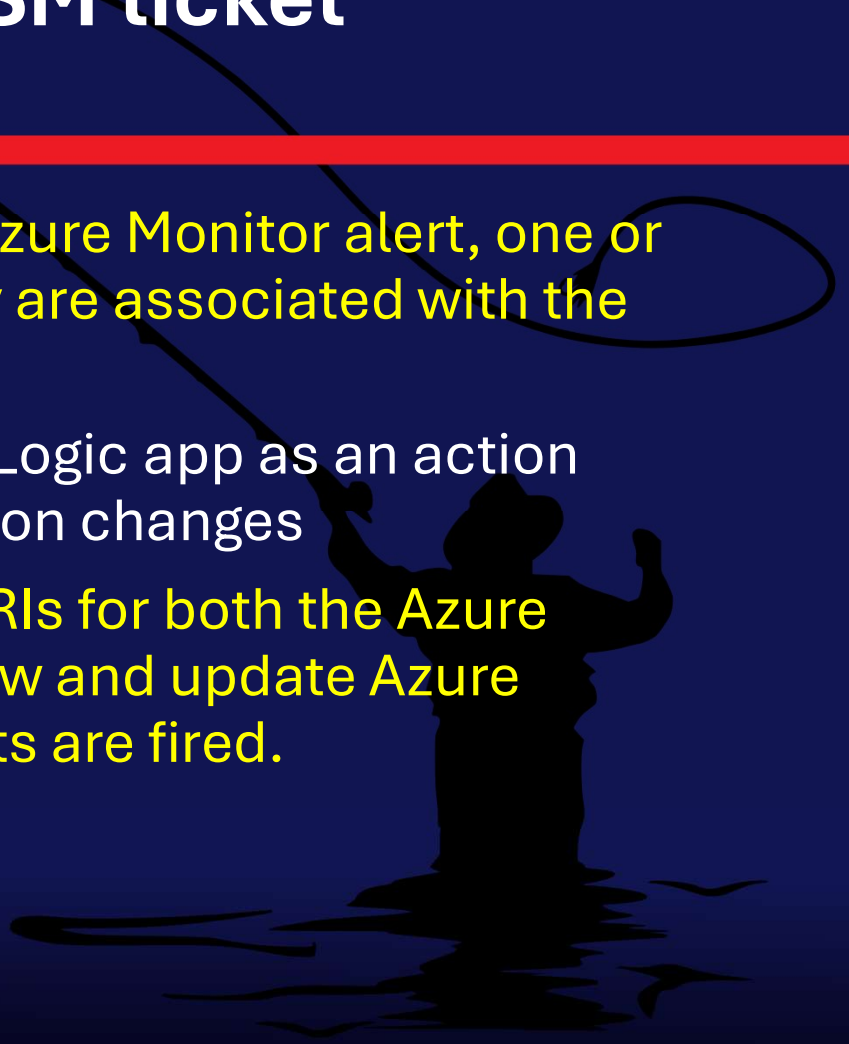
The overview page of a Logic app that is invoked by an Azure Monitor action group every time an Azure Monitor alert changes state




The Azure Monitor alert + ITSM ticket lifecycle
Every time a state change occurs in an Azure Monitor alert, one or more Action Group(s) can be fired if they are associated with the respective alert rule.
MMS

The Azure Monitor alert + ITSM ticket lifecycle

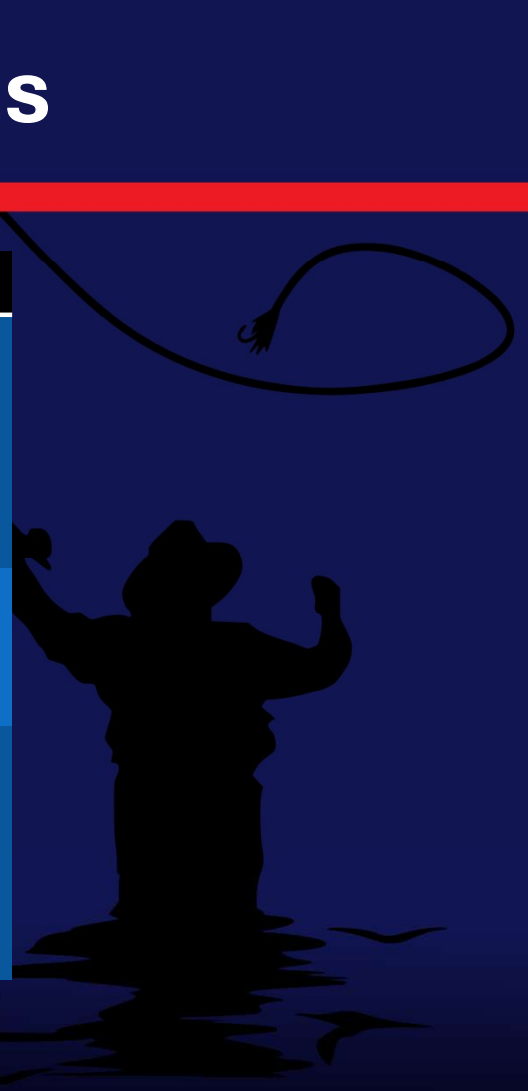
- Every time a state change occurs in an Azure Monitor alert, one or more Action Group(s) can be fired if they are associated with the respective alert rule.
- Create an Action Group that specifies a Logic app as an action task when an Azure Monitor alert condition changes
- The Logic app will use HTTP REST API URIs for both the Azure management API and the ITSM API to view and update Azure Monitor alerts in both systems after alerts are fired.



MMS

Azure Monitor alert control points

Control point	Possible values	Comments
Alert severity	• Sev0 (Critical) • Sev1 (Error) • Sev2 (Warning) • Sev3 (Informational) • Sev4 (Verbose)	Defined by the Alert rule, can't be changed after alert fires.
Alert condition	• Fired • Revolved	Set automatically by the system, can't be changed by the user
User response	• New • Acknowledged • Closed	Fully and only user-controlled. When changing user response, a comment can be optionally added to the alert history.



Azure Monitor alert metadata at play

Azure Monitor alert severity, alert condition, and user response are the moving parts we have to work with.

Azure Monitor alert signal types

The ITSM Logic app needs to accommodate the different types of Azure Monitor alerts that will arise from managing Windows and Linux servers with Azure Monitor.

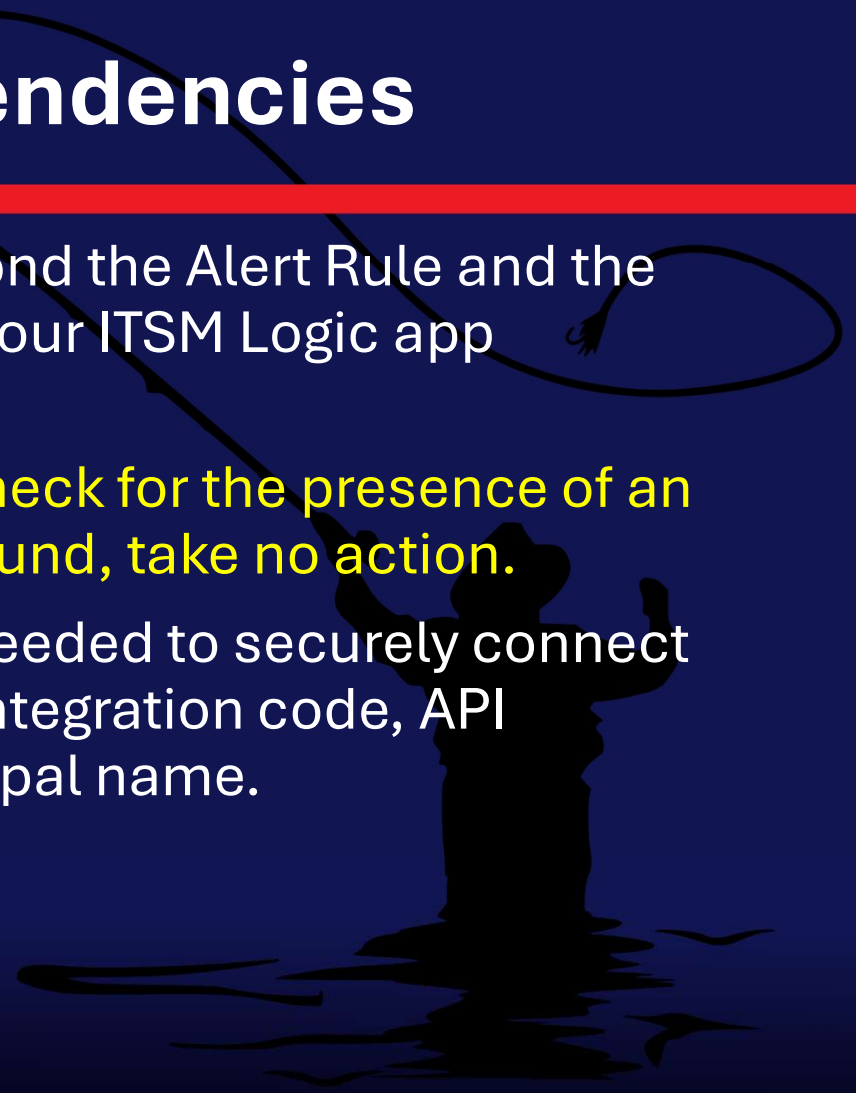
- Log alerts from Log Analytics query (examples: VMInsights performance, Event log records)
- Log alerts from Azure Resource Graph (ARG) query (example: Azure Arc server connection status)
- Metric alerts from Azure platform (example: Loss of Azure Monitor Agent heartbeat from a server)

Note: There are other types of Azure Monitor alerts, such as AppInsights *Smart detection* alerts and *activity* or *platform service* and *resource health* alerts. ITSM support for these types of Azure Monitor alerts is outside the scope of this server-focused solution. You could add such support by extending the Logic apps supplied with this solution or writing new but similar Logic apps dedicated to alerts not sourced from server-based monitoring.

Other Azure service dependencies

There are at least two Azure artifacts beyond the Alert Rule and the Action Group that should be included in your ITSM Logic app design:

- Azure Monitor Alert processing rules: Check for the presence of an enabled *alert suppression rule*, and if found, take no action.
- Azure Key Vault: Store the information needed to securely connect with your ITSM system such as the API integration code, API secret, and API username/service principal name.



MMS

Bi-directional support

- Most of the relevant actions in our scenario are generated from the Azure side, such as when an alert is first fired, and when/if the alert is changed to a Closed state by Azure Monitor.
- Not all alerts can be automatically closed by Azure Monitor—only alert rules that support a corrective threshold and with the “Automatically resolve alerts” setting enabled will ever transition from Fired to Resolved.
- However, when we consider that the ultimate source of truth for alert user responses is the ITSM ticket, we need to keep the Azure Monitor alert history synchronized with the ITSM ticket status. This is accomplished by staging a trigger action (or ‘automation rule’) within the ITSM application that launches a webhook when an Azure Monitor ITSM ticket is marked Completed in the ITSM system.

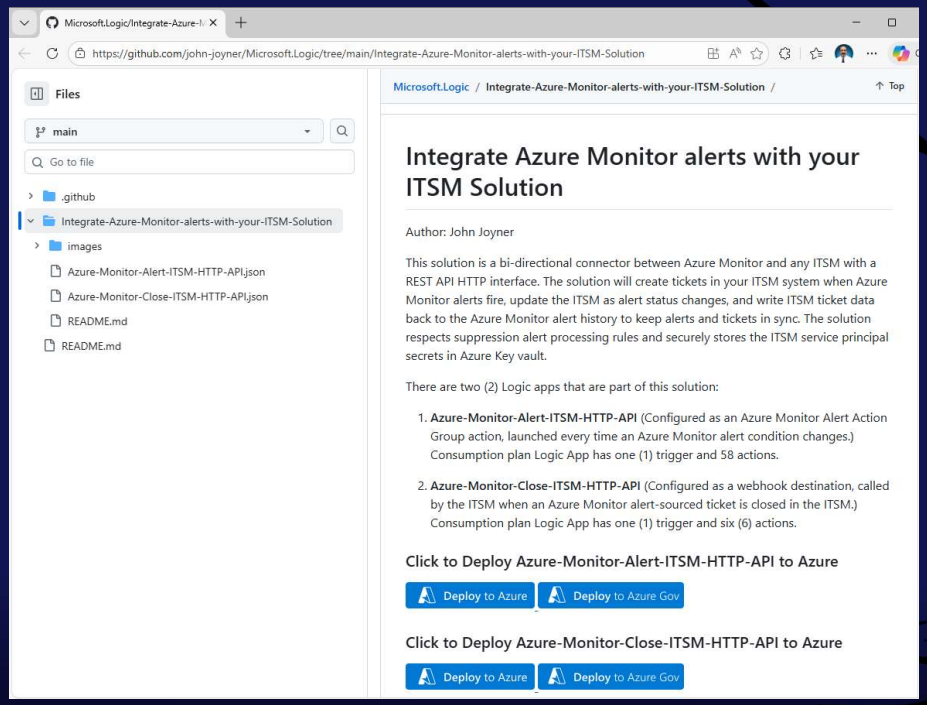
Authoring your Azure Monitor ⇔ ITSM system Logic apps

Each organization’s environment and their unique business needs and ITSM system(s) will drive customized Logic apps in every case.

MMS

Authoring your Azure Monitor ⇔ ITSM system Logic apps

Framework: https://github.com/john-joyner/Microsoft.Logic/tree/main/Integrate-Azure-Monitor-alerts-with-your-ITSM-Solution



Authoring your Azure Monitor ⇔ ITSM system Logic apps

Customize for your ITSM system:

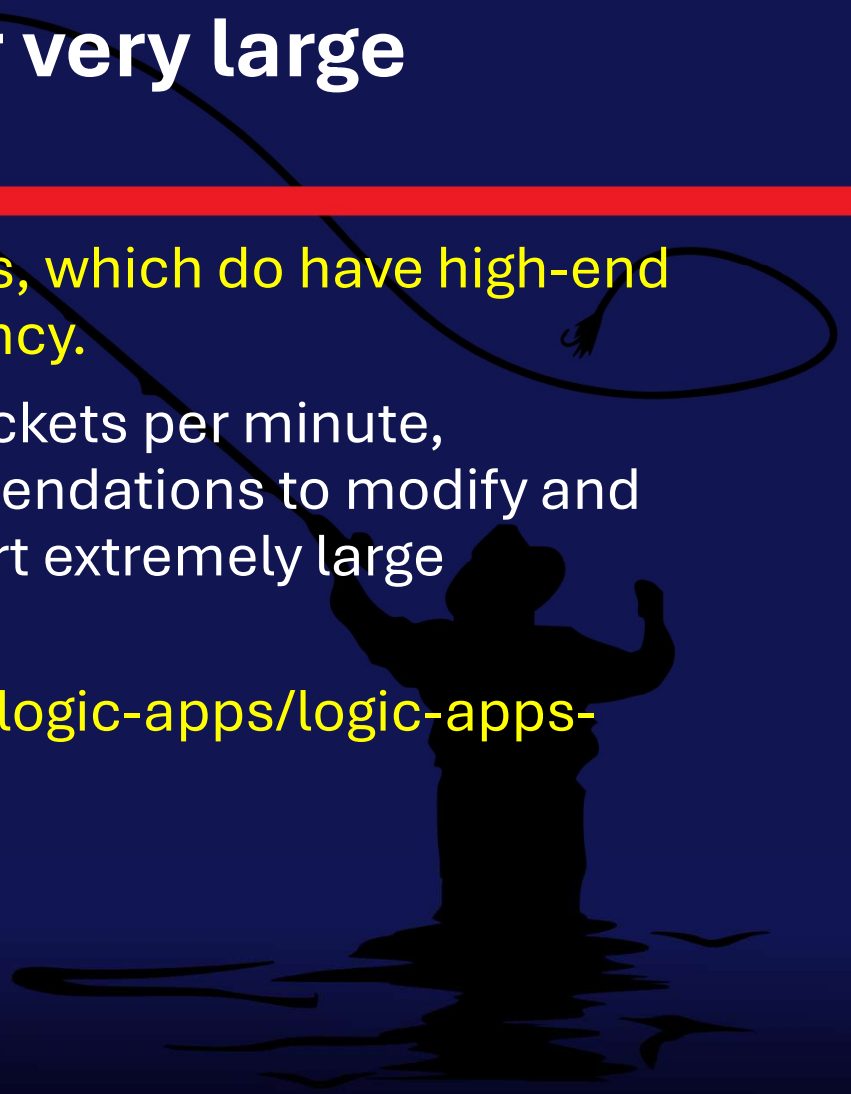
ITSM Product	URL to REST API documentation
ServiceNow	https://www.servicenow.com/docs/bundle/zurich-api-reference/page/build/applications/concept/api-rest.html
Zendesk	https://developer.zendesk.com/api-reference/
JIRA software	https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro/
Freshdesk	https://developer.freshdesk.com/api/v1/
ManageEngine	https://www.manageengine.com/api/
Autotask	https://www.autotask.net/help/DeveloperHelp/Content/APIs/REST/REST_API_Home.htm
BMC Helix	https://docs.bmc.com/xwiki/bin/view/Helix-Common-Services/Other/BMC-Helix-Subscriber-Information/helixsubscriber/Getting-started/Integrations/REST-APIs/
Ivanti	https://help.ivanti.com/ht/help/en_US/ISM/2022/admin/Content/Configure/API/RestAPI-Introduction.htm

Capacity considerations for very large environments

This solution is based on Azure Logic apps, which do have high-end limits regarding throughput and concurrency.

If your environment might exceed 1,000 tickets per minute, consult this link for best practice recommendations to modify and extend Logic app configurations to support extremely large compute demands:

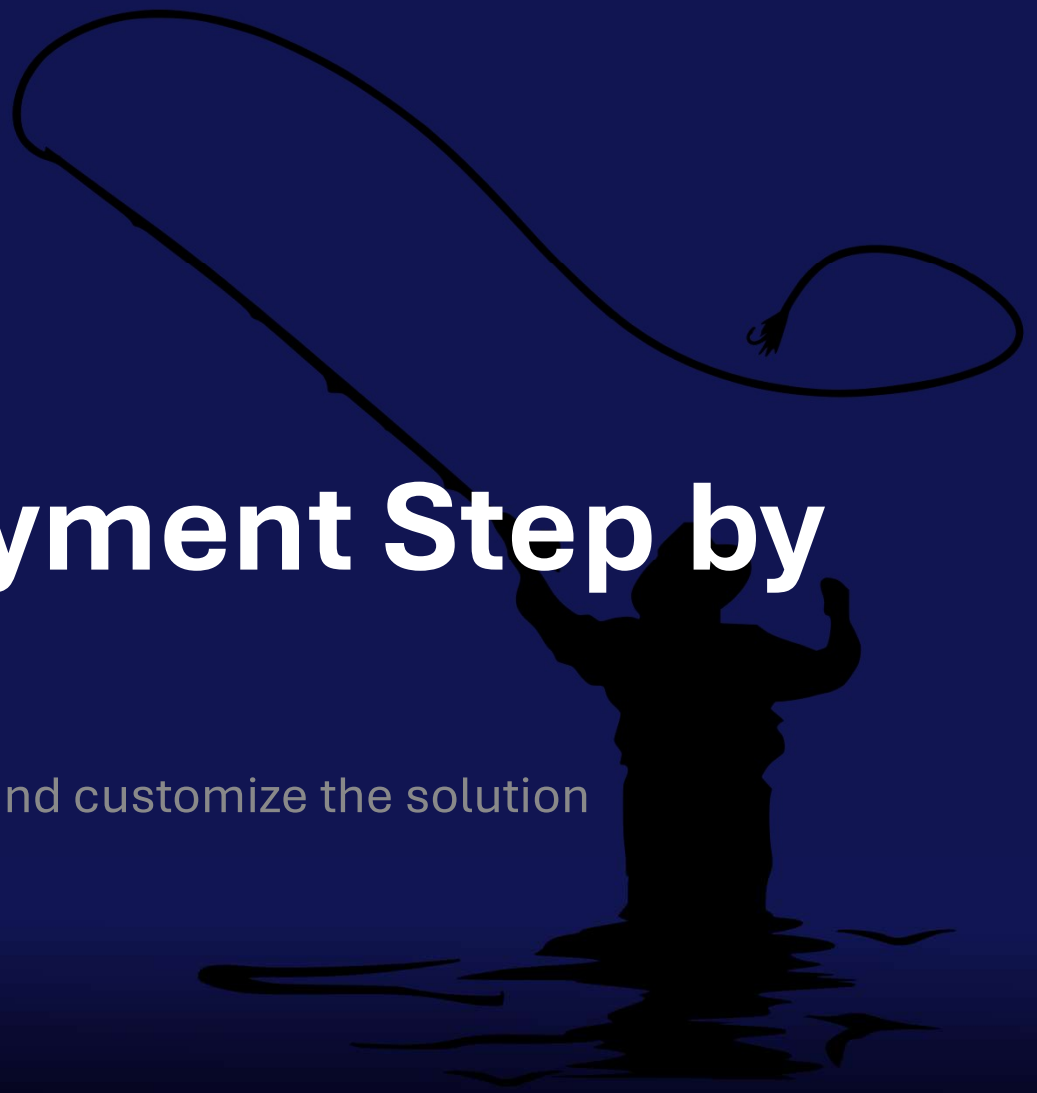
https://learn.microsoft.com/en-us/azure/logic-apps/logic-apps-limits-and-config



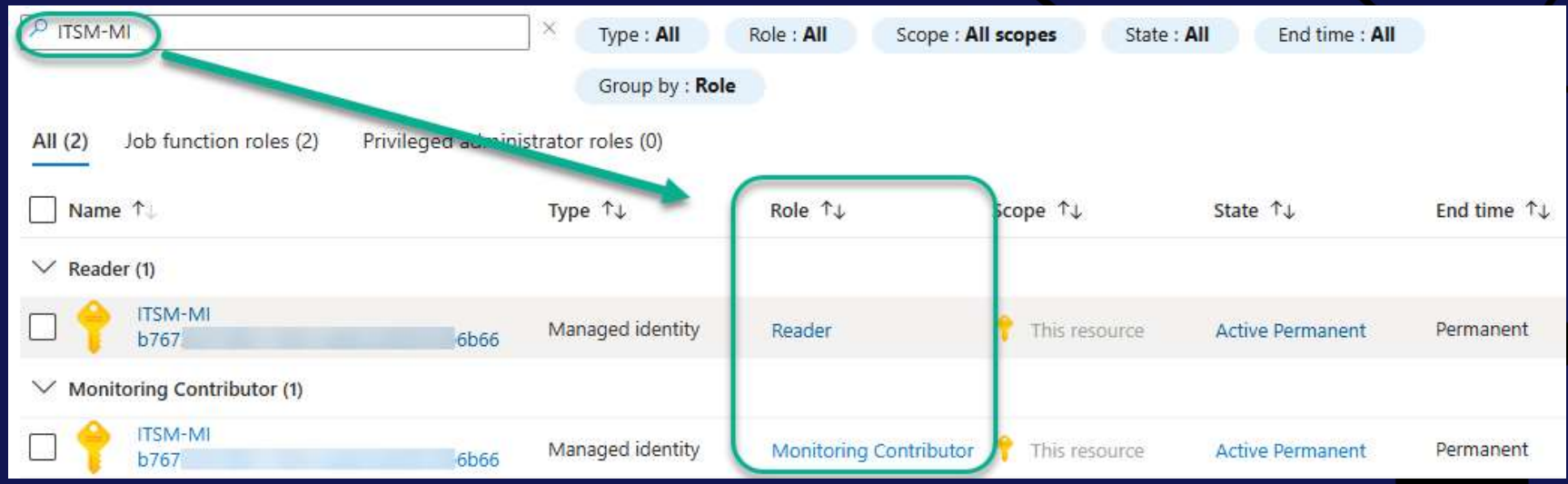
MMS

Solution Deployment Step by Step

Recommended project steps to deploy and customize the solution



1 – Create a service principal in the ITSM application and user-assigned managed identity in Azure

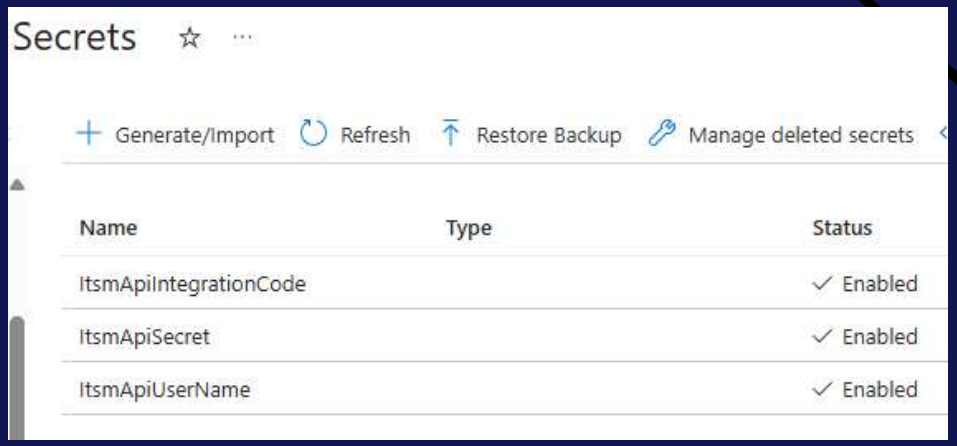


Azure RBAC roles in the subscription for the service principal (Managed identity) that will update Azure Monitor alerts.

2 – Get data from the ITSM needed for API connection

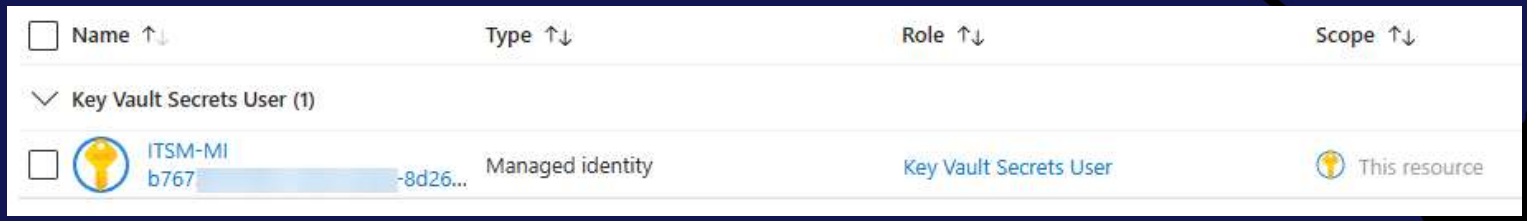
- Establish the correct URL for Azure Logic app to connect to the ITSM API, this is referred to as the ITSM Integration Code. An example might be https://MyCompany.MyItsm.com/api.
- From your ITSM administrator, obtain the Username (service principal name or GUID) and password (secret) needed to read and post data to the ITSM API.
- Obtain other parameters needed for ticket creation with the ITSM API. The Logic app assumes there is a *Company ID* and *Queue ID* associated with the place in the ITSM where tickets should be created. If your ITSM has additional or different parameters of this nature, take note of them and you will customize the ticket creation step in the Logic app later with this information.
- While the service principal on the Azure side is a Managed identity without an associated token, secret, or expiration date, the service principal created on the ITSM side may have an expiration date. If this is the case, be sure and create a scheduled maintenance task to renew the ITSM access token or secret before the expiration date.

3 – Create an Azure Key vault and populate it with necessary data and secret to connect to the ITSM API



Secrets needed by the Logic app to access the ITSM API are stored in Key vault.

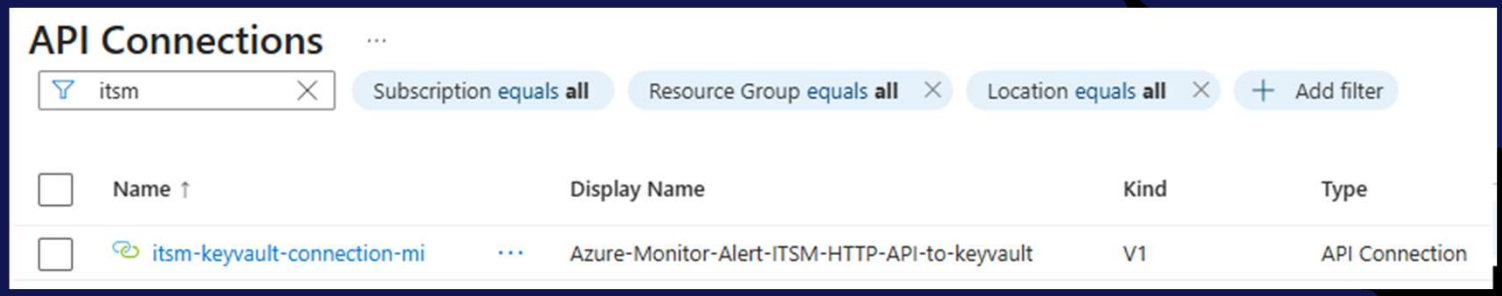
4 – Grant the Azure service principal RBAC permission to read secrets from the Key vault



Name	Type	Role	Scope
Key Vault Secrets User (1)			
ITSM-MI b767 -8d26...	Managed identity	Key Vault Secrets User	This resource

Verifying the Key Vault Secrets User role assigned to the user-assigned managed identity.

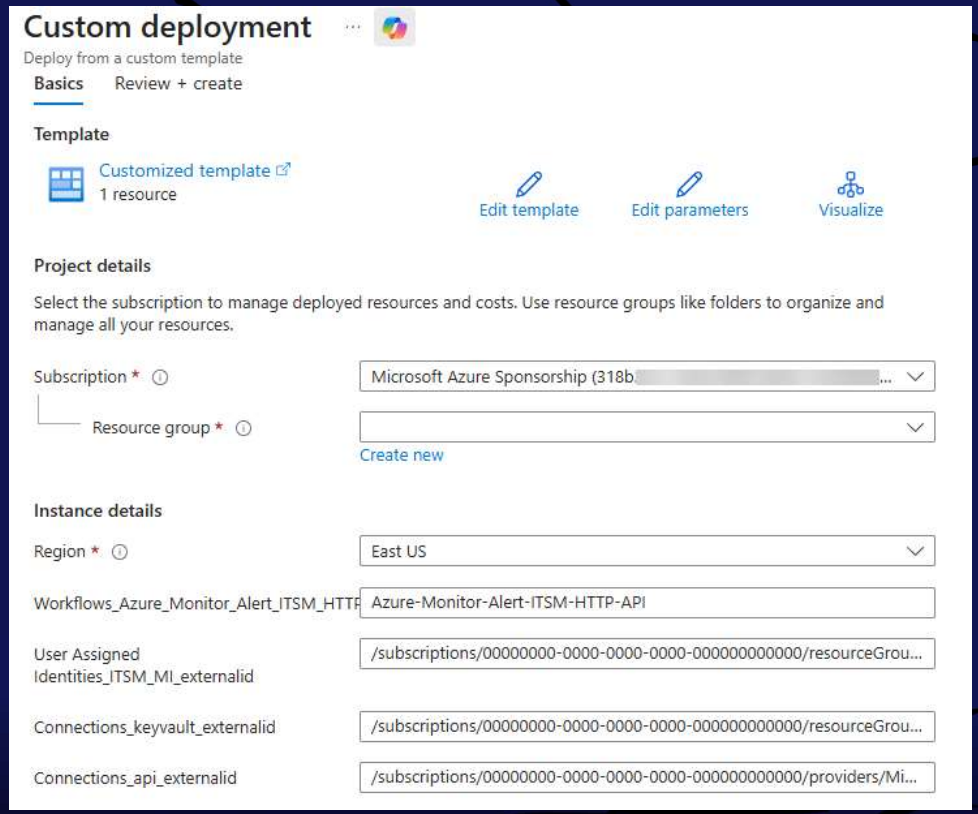
5 – Create the API connection the Logic app uses to connect to Key vault



API connection pre-created to use with the Logic app Azure-Monitor-Alert-ITSM-HTTP-API

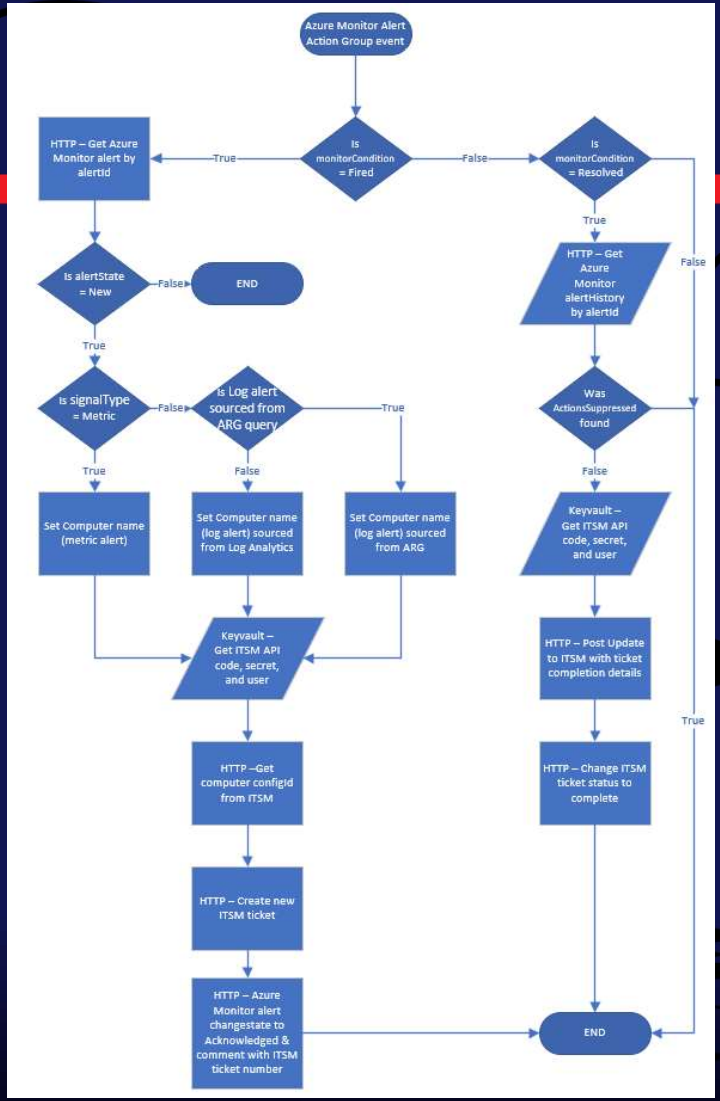
6 – Deploy the two (2) Logic apps from GitHub

Instance details of Azure_Monitor_Alert_ITSM_HTTP_API to be confirmed before creating the Logic app



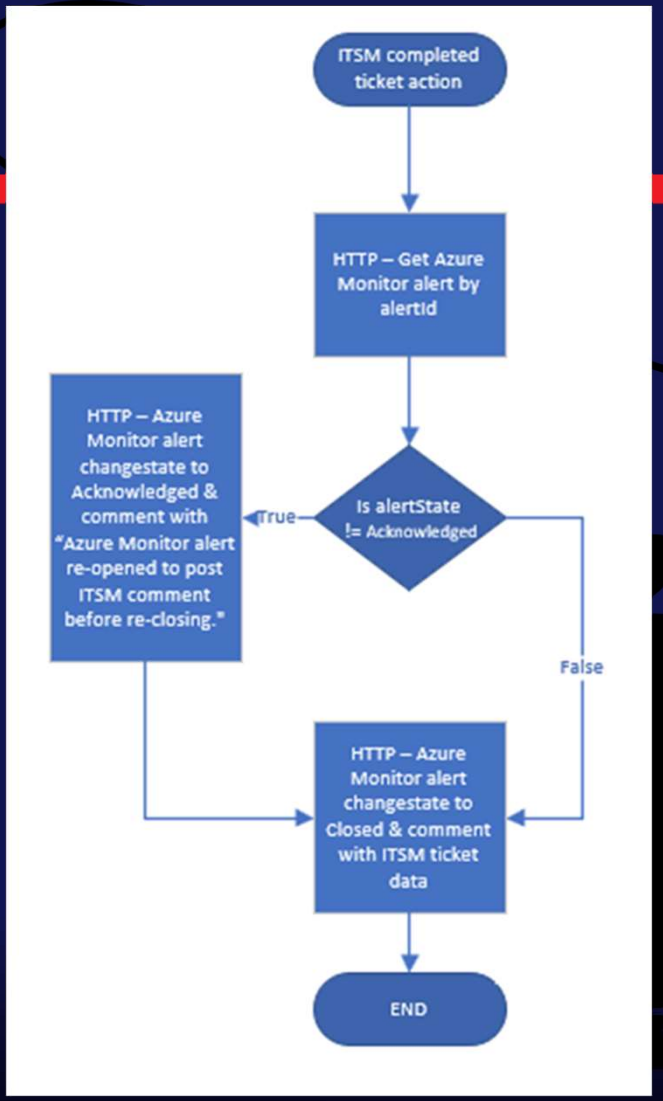
Logic app: Azure-Monitor-Alert-ITSM-HTTP-API

- This Logic app is staged in an Azure Monitor Alert Action Group to be invoked when an Azure Monitor alert changes condition (“Fired” and “Resolved”).
- The purpose of this Logic app is to interact with the ITSM system to open new ITSM tickets and change the Azure Monitor alert state from “New” to “Acknowledged”, then update corresponding ITSM tickets when Azure Monitor alerts are auto-resolved.
- Before creating the ticket, the Logic app retrieves asset information about the Computer that is the subject of the alert, so that the ticket can be opened under the context of the affected server.

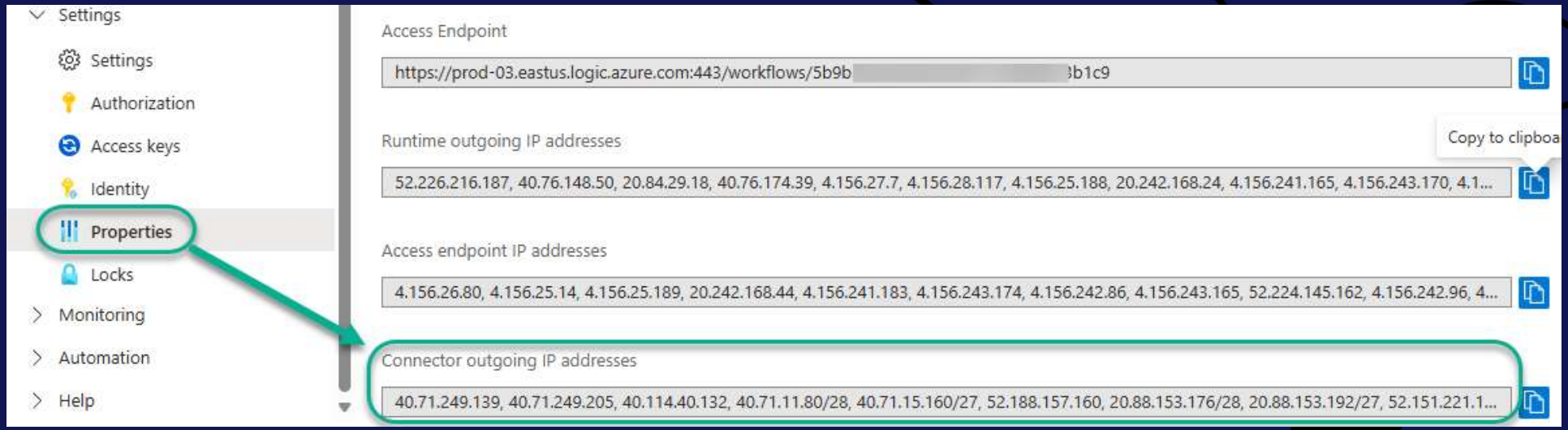


Logic app: Azure_Monitor_Close_ITSM_HTTP_API

- The webhook URL to this Logic app is staged in the ITSM system's automation to be invoked when an ITSM ticket sourced from Azure Monitor is marked completed.
- The main purpose of this Logic app is to keep the [Azure Monitor Alert "Closed"] <-> [ITSM Ticket "Completed"] status in sync.
- The Logic app also comments the alert history with the ITSM ticket number and other monitoring metadata information desired to document in the alert.



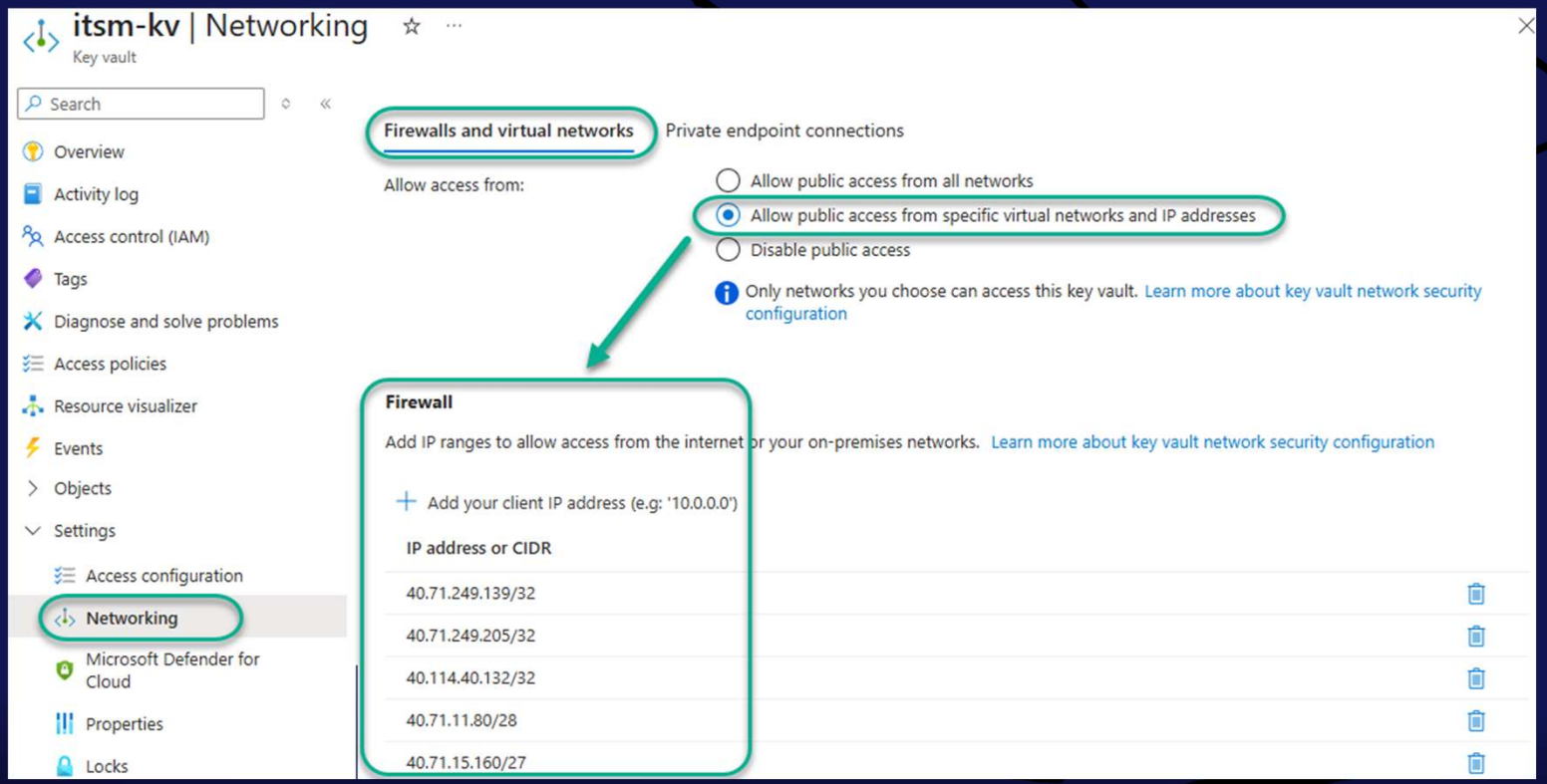
7a – Add the outbound IP ranges of the Logic app Azure-Monitor-Alert-ITSM-HTTP-API to the Firewall of the Key vault



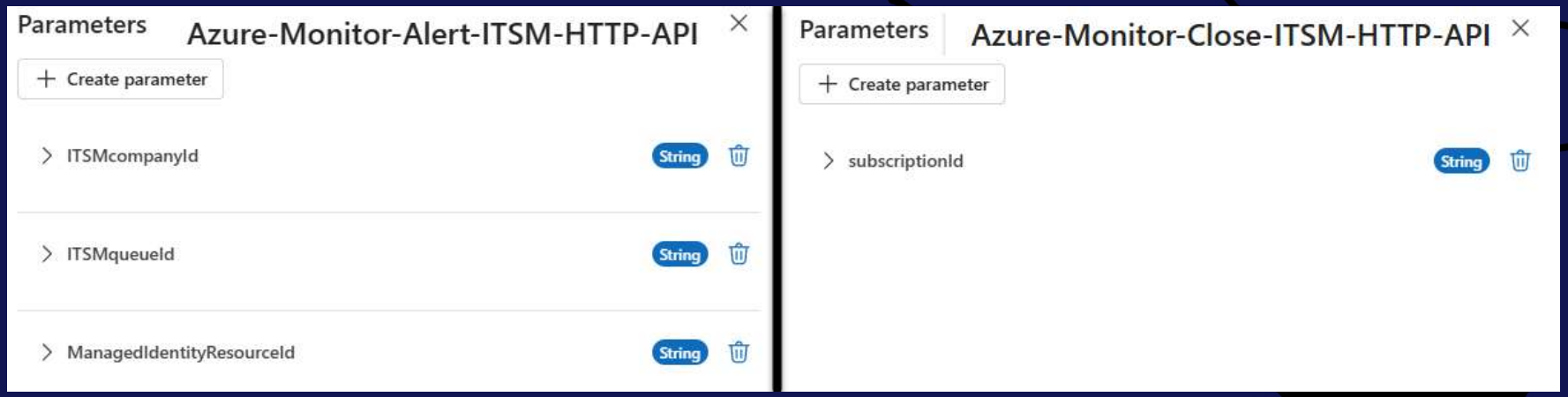
Locate the Connector outgoing IP addresses in the Logic app Azure-Monitor-Alert-ITSM-HTTP-API

7b – Add the outbound IP ranges of the Logic app Azure-Monitor-Alert-ITSM-HTTP-API to the Firewall of the Key vault

Adding your own public IP and the Logic app's Connector outgoing IP addresses to the local firewall of the Key vault.



8 – Customize the Logic apps' parameters to fit your organization



Parameters for the Azure Monitor "Alert" (left) and "Close" (right) ITSM API Logic apps.

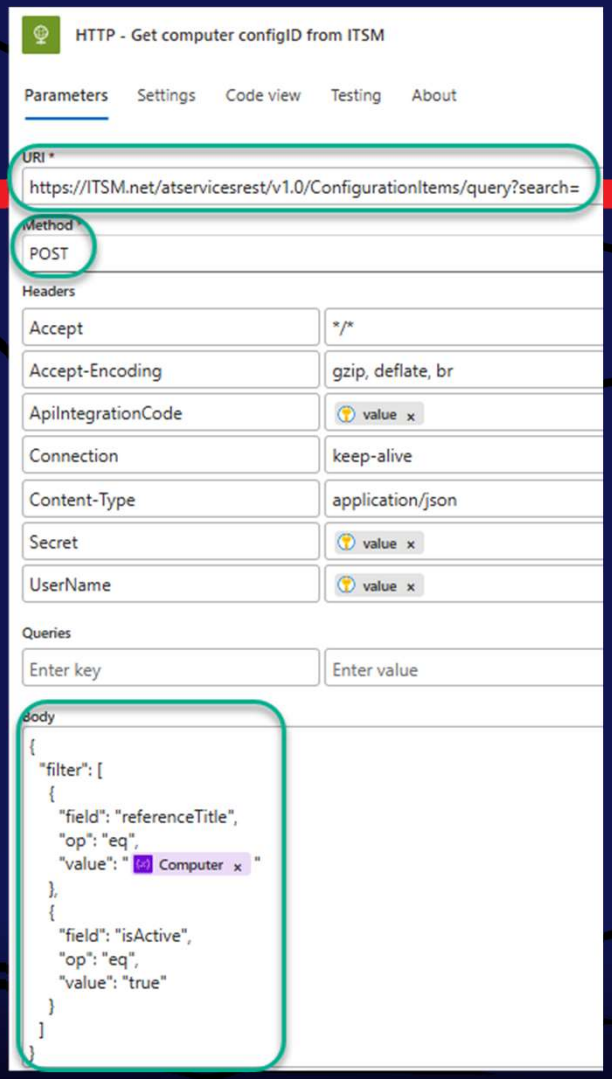
Note: The Logic apps deploy in a Disabled state and with Access Control Configuration *Allowed inbound IP address* settings enabled to comply with a Zero Trust model. When you are ready, Enable the Logic apps and customize the Access Control Configuration of both Logic apps so you can begin to test them.

9a – Customize the Logic apps' ticket creation and ticket update "HTTP" task outputs to match expected and desired data fields relevant to your ITSM application

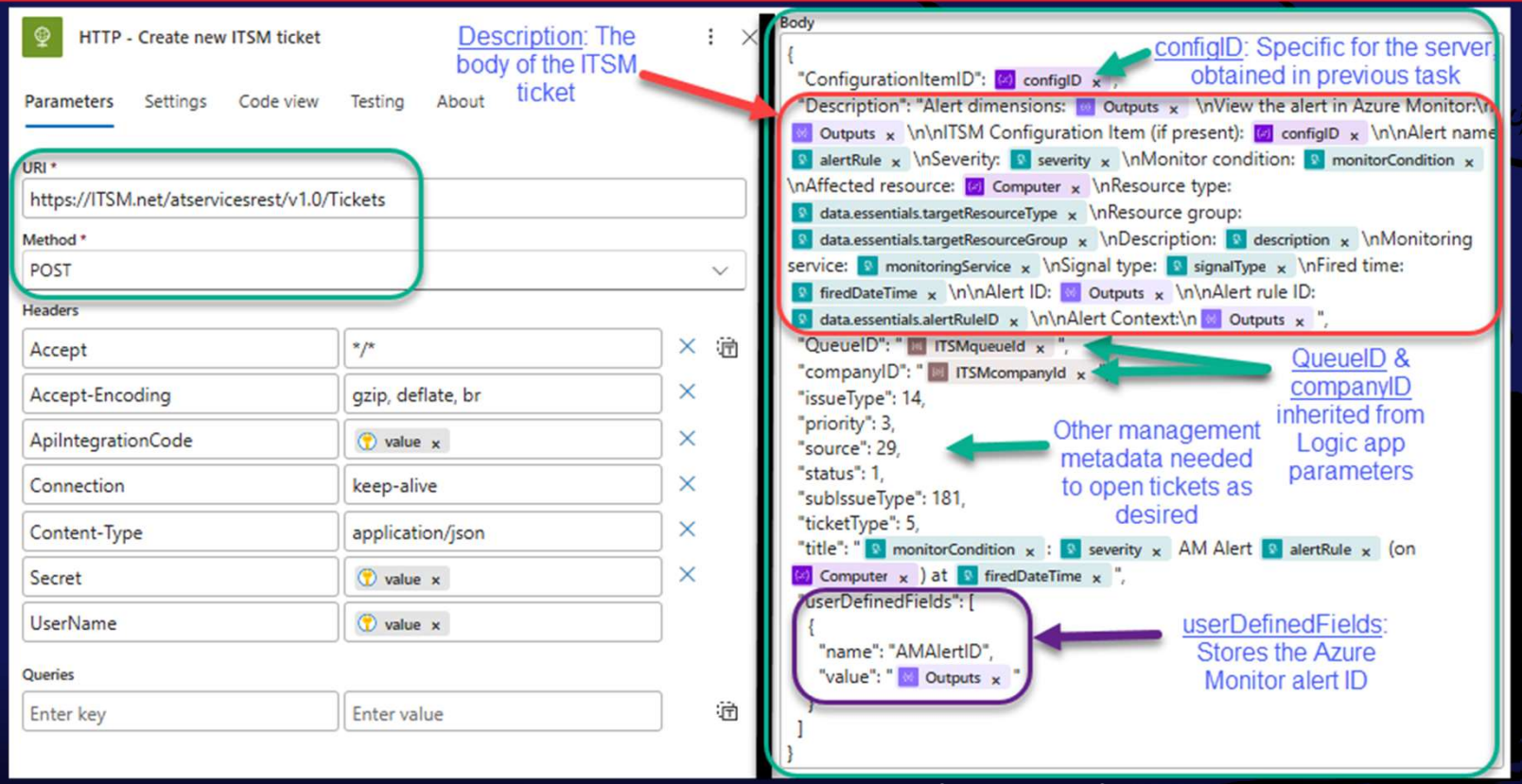
Logic app task **HTTP – Get computer configID from ITSM** pre-fetches the computer's configuration from the ITSM.

You will need to customize this task with the following information specific to your ITSM application:

- URI and Method: This will vary by your ITSM application's design. The default example is a POST command to the ITSM API running a query search based on the computer name.
- Body: Again, varies by your ITSM's configuration schema. The default example provides the Computer name in the Azure Monitor alert as the "referenceTitle" for the query and checks that the configuration is active in the ITSM.



9b – Customize the Logic apps' ticket creation and ticket update "HTTP" task outputs to match expected and desired data fields relevant to your ITSM application



Creating the ITSM ticket using all the collected and processed information from Azure Monitor and the ITSM.

9c – Behold the ticket!

The body of an ITSM ticket sourced from Azure Monitor and created by the Logic app using the ITSM's API.

Fired: Sev2 AM Alert Computer Late Heartbeat Alert (on [redacted]) at 2025-10-22T18:02:34.7060477Z

Created 10/22/2025 01:02 PM (5d 2h 25m ago) - Azure Monitor Integrator

Description

Alert dimensions: "{"name":"Computer","value":"[redacted]"}"
View the alert in Azure Monitor:
https://portal.azure.com/#view/Microsoft_Azure_Monitoring_Alerts/AlertDetails.ReactView/alertId/%2fsubscriptions[redacted]%2fresourceGroups%2[redacted]2fproviders%2fMicrosoft.AlertsManagement%2falerts%2f35b523fd-5c6b-4e64-bbc7-497c15cff000

ITSM Configuration Item (if present): 9394

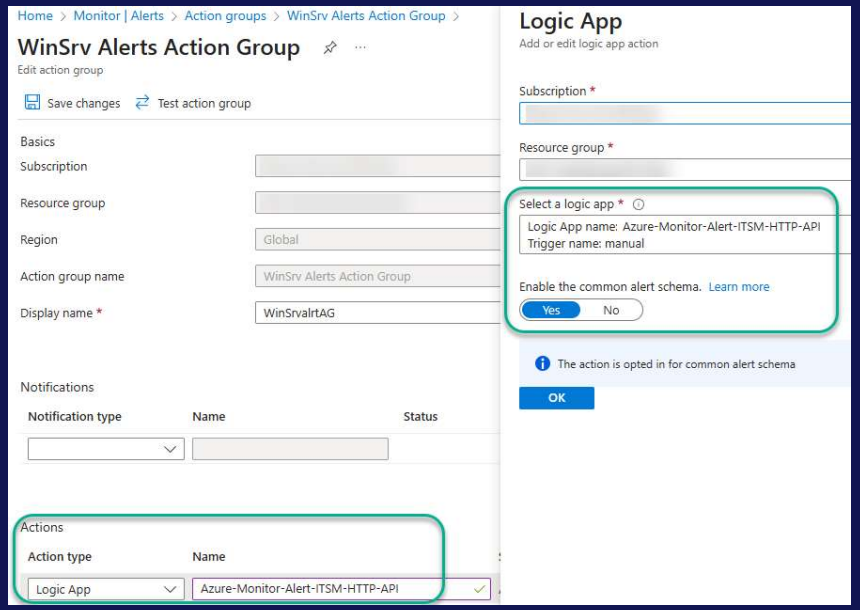
Alert name: Computer Late Heartbeat Alert
Severity: Sev2
Monitor condition: Fired
Affected resource: [redacted]
Resource type: microsoft.operationalinsights/workspaces
Resource group: [redacted]
Description: Metric query using the Heartbeat table, which should have a heartbeat record every minute from each machine. The threshold for stale heartbeat has been exceeded. Investigate the status of the computer(s) without recent heartbeat.
Monitoring service: Platform
Signal type: Metric
Fired time: 2025-10-22T18:02:34.7060477Z

Alert ID: 35b523fd-5c6b-4e64-bbc7-497c15cff000

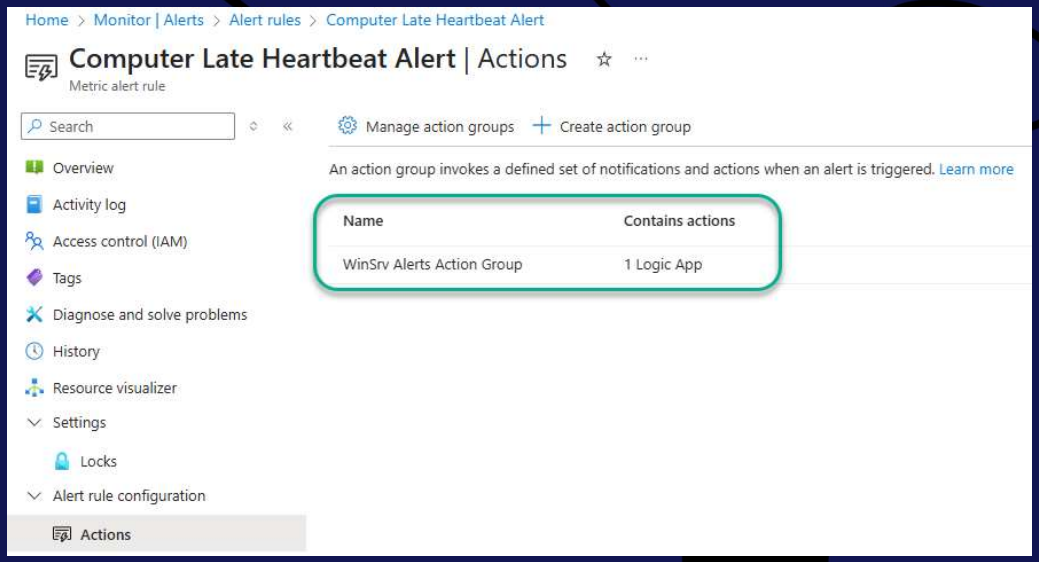
Alert rule ID: /subscriptions/[redacted]/resourceGroups/[redacted]/providers/microsoft.insights/metricAlerts/Computer Late Heartbeat Alert

Alert Context:
{"properties":null,"conditionType":"SingleResourceMultipleMetricCriteria","condition":{"windowSize":"PT5M","allOf":[{"metricName":"Heartbeat","metricNamespace":"Microsoft.OperationalInsights/workspaces","operator":"LessThan","threshold":"4","timeAggregation":"Count","dimensions":

10 – Create an Action group that includes the Logic app Azure-Monitor-Alert-ITSM-HTTP-API as an action and associate the Action group with one or more Azure Monitor alert rules



Associating the Logic app with the Windows Servers alert action group



In this configuration, the WinSrv Alerts Action Group will be invoked when the Azure Monitor alert fires, and again when and if the alert is resolved by the system.

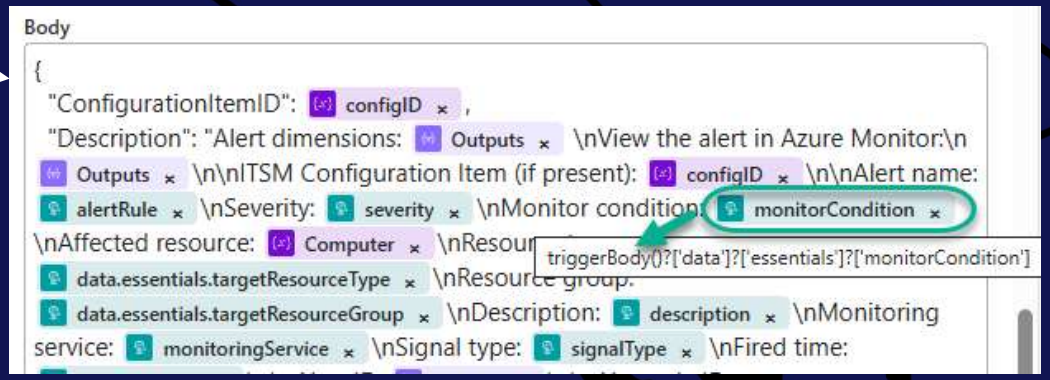
11a – Cause Azure Monitor alerts to fire and observe Logic app behavior

Study the raw output of the first task in the Logic app, which is the trigger task and contains a dump of the complete alert.

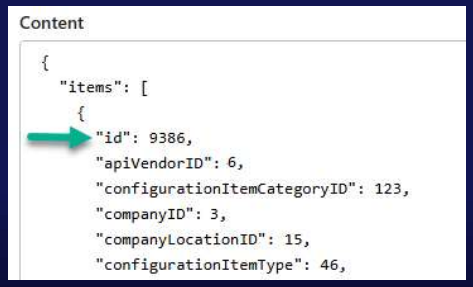
This is where we are starting from: The complete Azure Monitor alert ‘dump’. The rest of the Logic app takes this data and filters and transforms the raw data into formats that match up with the ITSM ticket schema so we can create an ITSM ticket using the ITSM API.

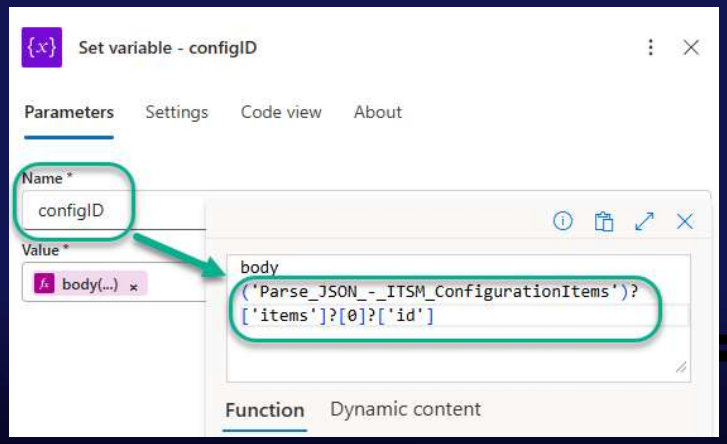
11b – Creating inputs to the ITSM that will work

Technique #1 – Filtering JSON by using nested property access



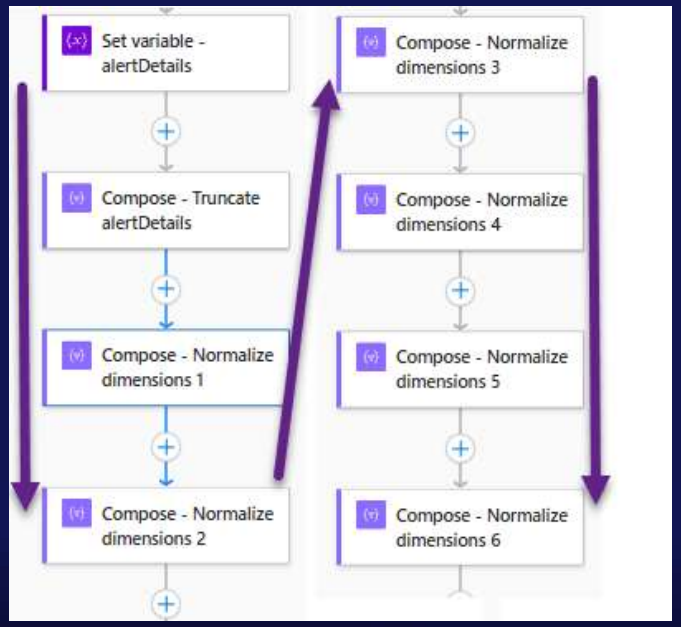
Technique #2 – Filtering JSON by accessing elements within an array based on their index values





11c – Creating inputs to the ITSM that will work

Technique #3 – Filtering JSON by using a series of Compose tasks



Logic app task name	Expression	Result
Set variable – alertDetails	triggerBody()?['data']?['alertContext']	Brings all the raw alertContext JSON array into the variable
Compose – Truncate alertDetails	first(split(variables('alertDetails'), 'linkToSearchResultsUI'))	Drops the text after a certain text string
Compose – Normalize dimensions 1	last(split(outputs('Compose_-_Truncate_alertDetails'),'dimensions'))	Drops the text before a certain text string
Compose – Normalize dimensions 2	first(split(outputs('Compose_-_Normalize_dimensions_1'),'metricValue'))}	Drops the text after a certain text string
Compose – Normalize dimensions 3	replace(outputs('Compose_-_Normalize_dimensions_2'), '\', '')	Replaces undesired text string with a new text string
Compose – Normalize dimensions 4	replace(outputs('Compose_-_Normalize_dimensions_3'), ':[', '')	Strips out undesired text string if present (leading ":[")
Compose – Normalize dimensions 5	replace(outputs('Compose_-_Normalize_dimensions_4'),']','')	Strips out undesired text string if present (trailing "],")
Compose – Normalize dimensions 6	replace(outputs('Compose_-_Normalize_dimensions_5'),',"type":null,"values":null,"operator":null',')	Strips out undesired text string if present in metric log alerts.

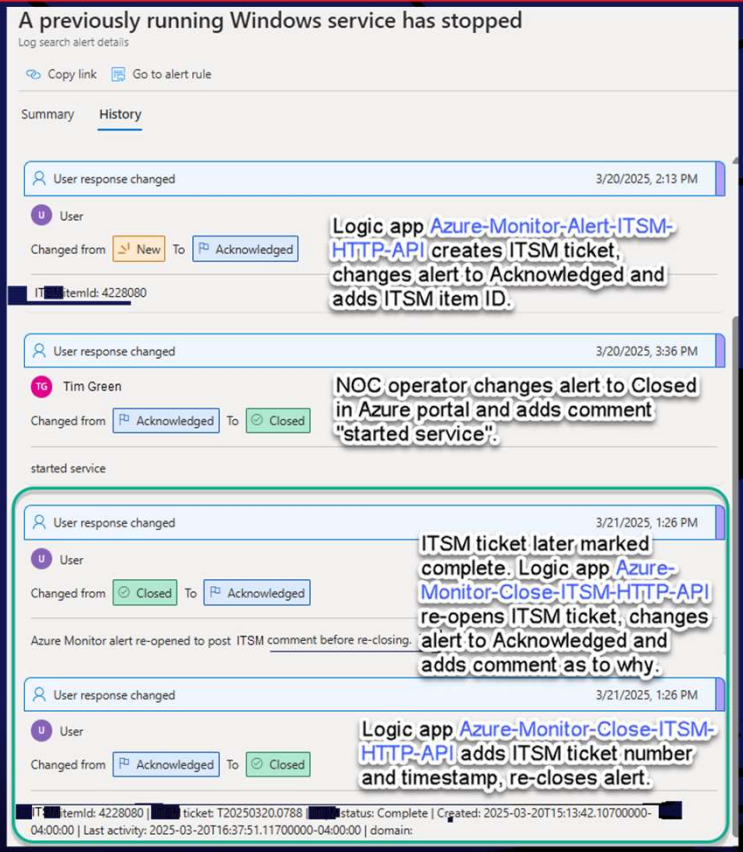
12 – For each of the Log Analytics, Azure Resource Graph, and Metric alert types, continue to customize the text transformation tasks of Logic app Azure-Monitor-Alert-ITSM-HTTP-API until ITSM tickets are correctly created and updated in all scenarios

- Azure Monitor alerts on servers are produced by Azure Monitor alert rules, and those alert rules fall into several categories that have slightly varying schemas. Make you test fire all the types of alerts you expect to encounter in production while customizing the Logic app.
- The several places in the Logic app where text transformations are taking place are completely open to your revision and refactoring using whatever Logic app tasks and expressions work for you.
- This is an iterative, trial and error process, repeated for each alert type. Every time the Logic app reports a failure, open the Run history and research why a particular step failed or didn't produce the expected outcome.
- As you customize the Logic app to match your alerts and ITSM, the more steps will execute successfully—until the entire Logic app reports success and you validate in your ITSM system that tickets are being created to your satisfaction and that of your NOC operators.

13a - Configure your ITSM to launch the webhook URL of the Logic app Azure-Monitor-Close-ITSM-HTTP-API for each Azure Monitor ticket that is marked completed in the ITSM

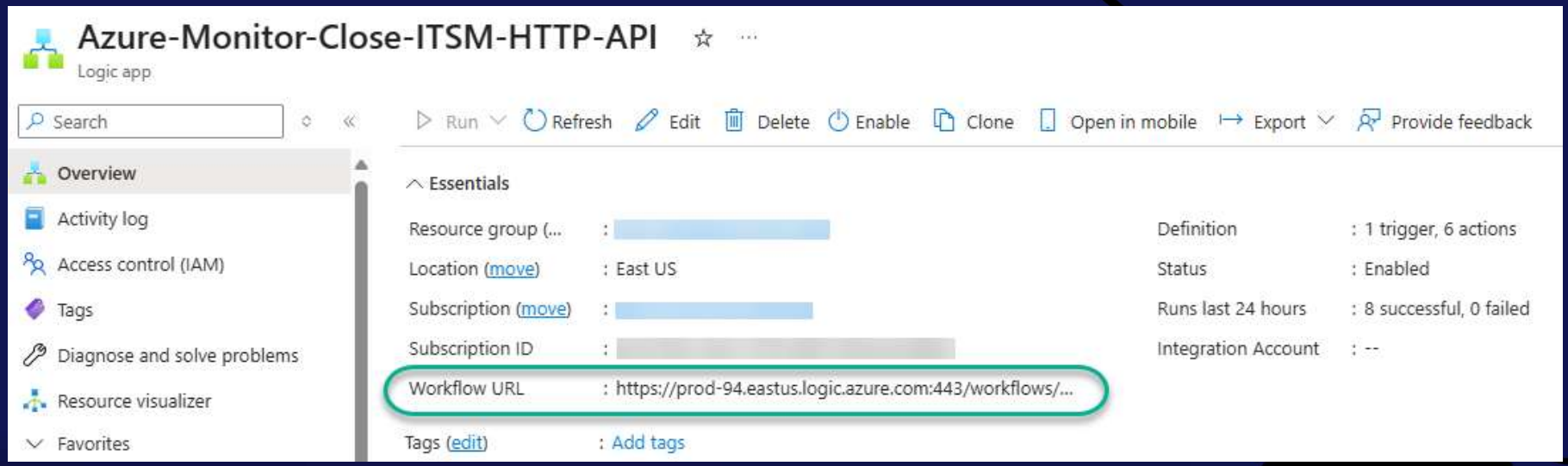
- The Logic app has one purpose—to change the user response of the Azure Monitor alert to *Closed* when the ticket is completed in the ITSM, and to record the ITSM ticket number, timestamp, and other audit data in the body of the closed alert history.

- If the alert is already in Closed status, the status is temporarily flipped back to Acknowledged so that the alert can be re-closed with the ITSM ticket audit trail in the comment history.



13b - Staging Azure-Monitor-Close-ITSM-HTTP-API to be invoked by the ITSM

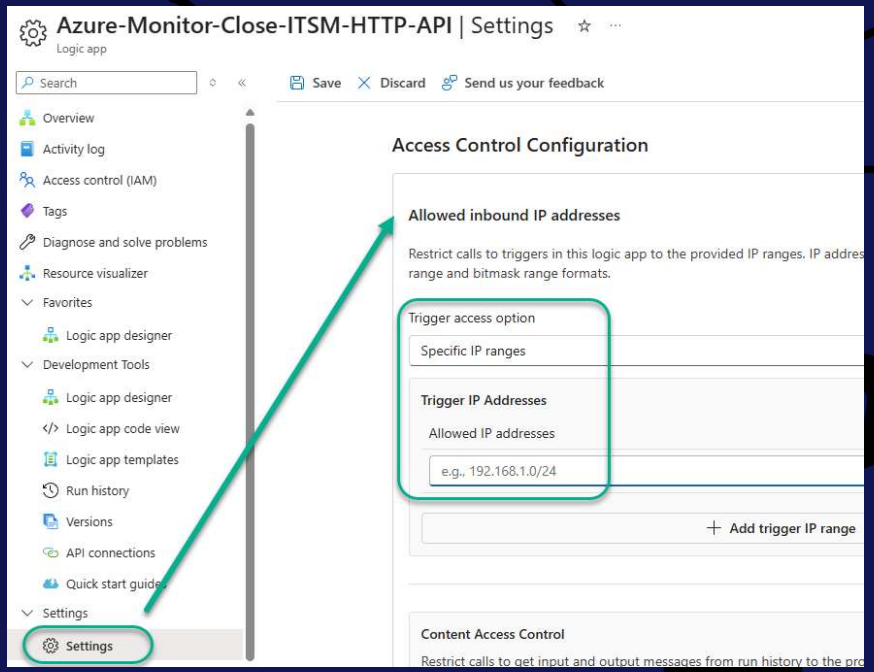
Obtaining the webhook destination for the Azure-Monitor-Close-ITSM-HTTP-API Logic app to provide to the ITSM admin



13c – (Optionally) Restrict access to invoke the Close webhook

- The default configuration of the consumption Logic apps in this solution *does* allow any Internet IP to initiate a webhook call if an unauthorized person learned the exact Workflow URL of your Logic app. If this is not satisfactory, you have several options to restrict this access.
- If you can define which ITSM public IPs will be calling the webhook URL, you can enter those IPs in the Allowed inbound IP addresses -> Trigger IP Addresses list of the Access Control Configuration of the Logic app **Azure-Monitor-Close-ITSM-HTTP-API**

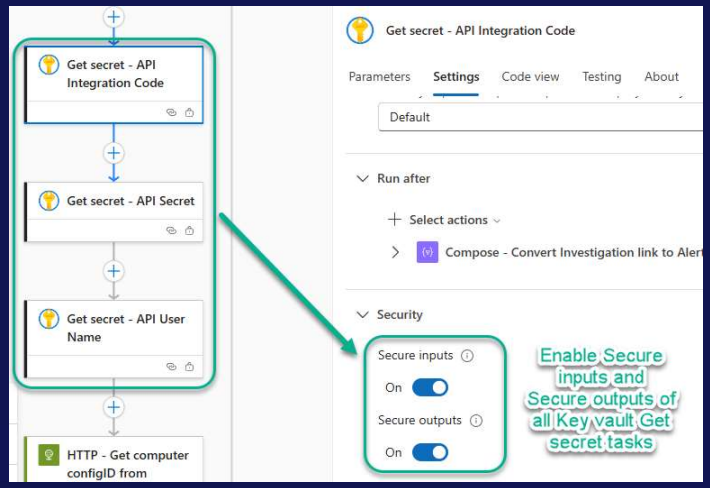
Note: To comply with a zero-trust model, the Access Control Configuration *Allowed inbound IP address* settings are enabled by default with arbitrary sample data. Customize the Access Control Configuration of both Logic apps so you can begin to test them.

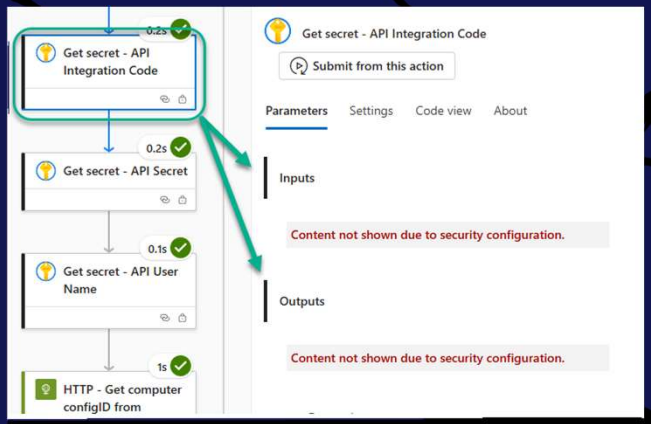


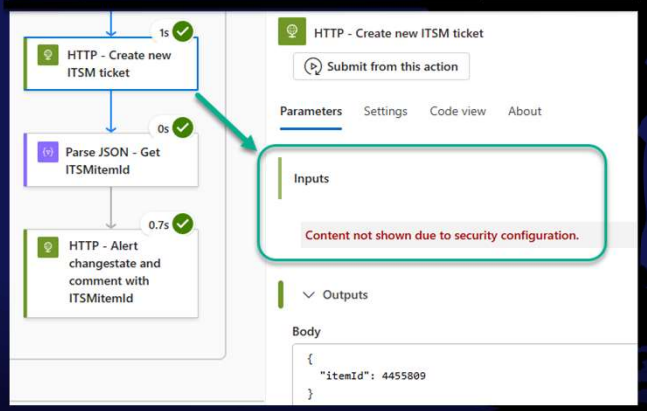
Advanced security options: https://learn.microsoft.com/en-us/azure/logic-apps/logic-apps-securing-a-logic-app

14 – Enable Secure inputs and outputs for the Key vault tasks in the Logic app Azure-Monitor-Alert-ITSM-HTTP-API

With Key vault tasks secured, all Logic app tasks involving secrets from Key vault are masked.

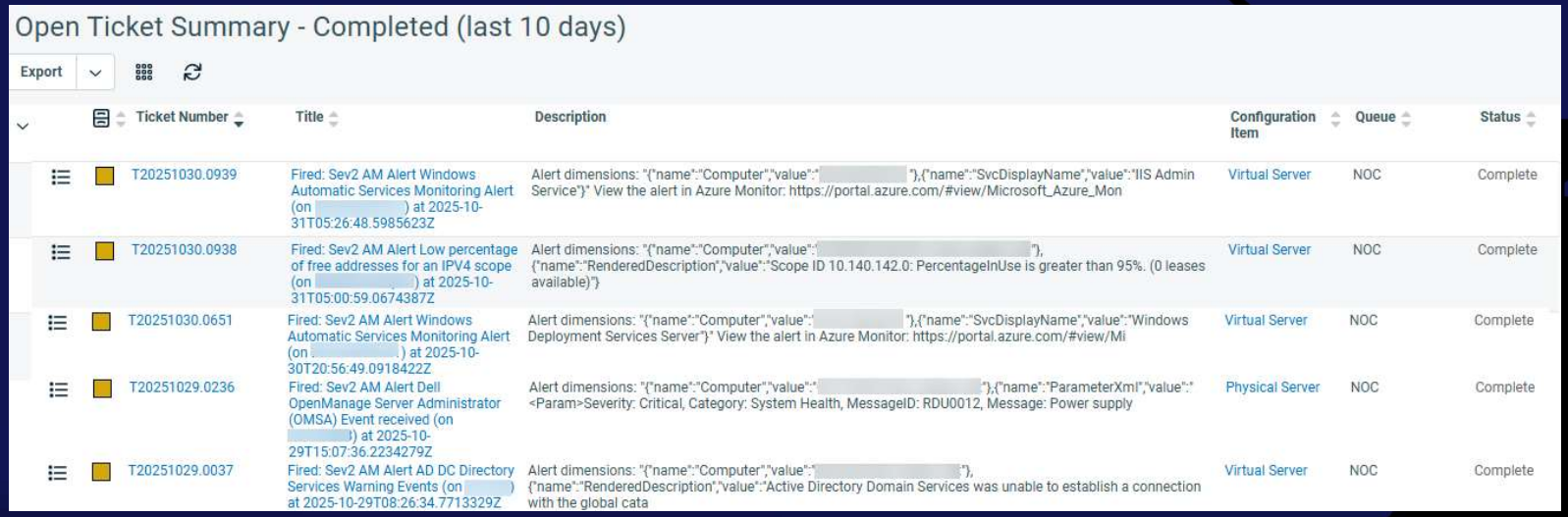






The desired end result: What your ITSM ticket view might look like when fully integrated with Azure Monitor in your environment.

Azure Monitor alerts viewed in the ITSM completed ticket queue: Azure Monitor tightly integrated with your ITSM



Open Ticket Summary - Completed (last 10 days)

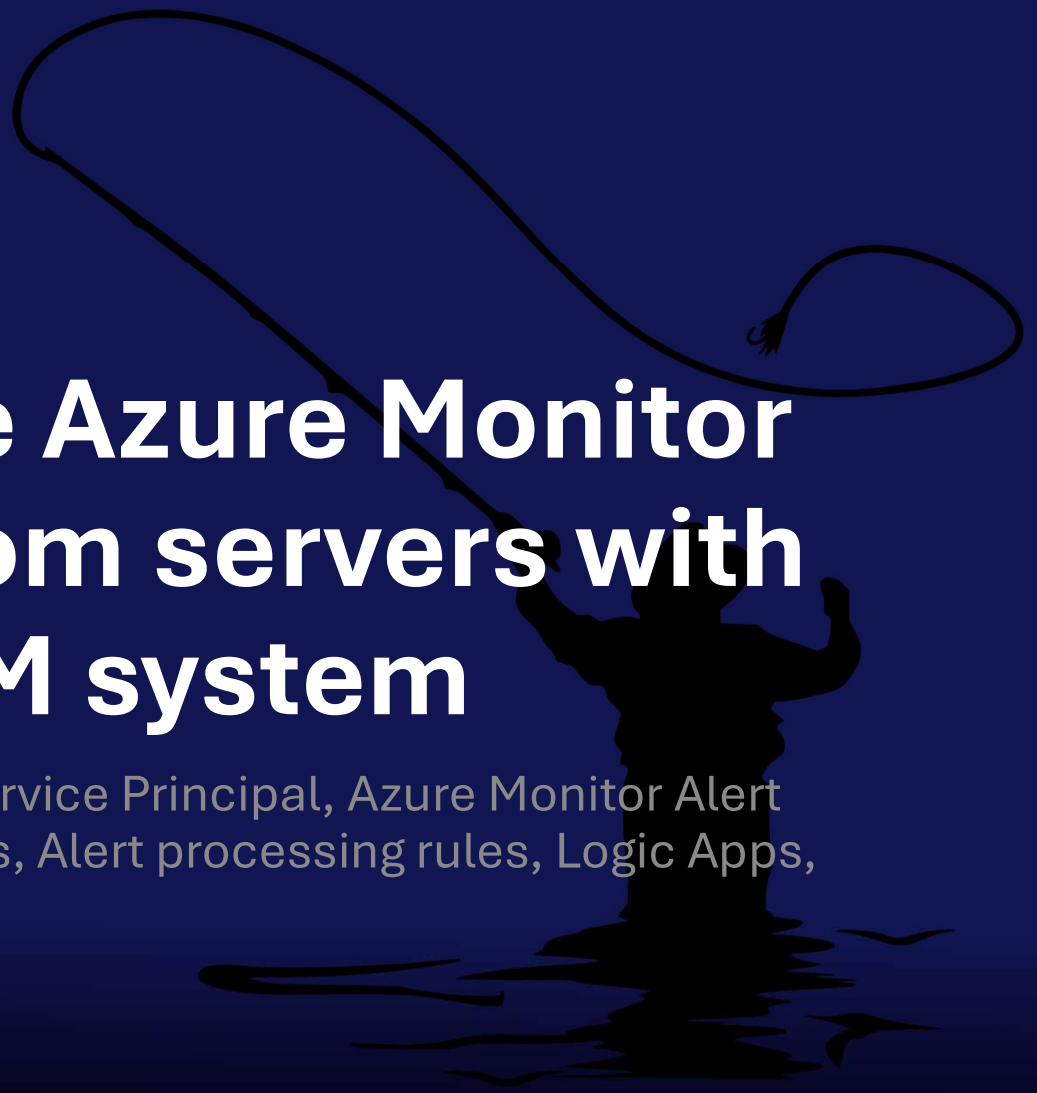
Export

Ticket Number	Title	Description	Configuration Item	Queue	Status
T20251030.0939	Fired: Sev2 AM Alert Windows Automatic Services Monitoring Alert (on) at 2025-10-31T05:26:48.5985623Z	Alert dimensions: "{"name":"Computer","value":" "},{"name":"SvcDisplayName","value":"IIS Admin Service"}" View the alert in Azure Monitor: https://portal.azure.com/#view/Microsoft_Azure_Mon	Virtual Server	NOC	Complete
T20251030.0938	Fired: Sev2 AM Alert Low percentage of free addresses for an IPV4 scope (on) at 2025-10-31T05:00:59.0674387Z	Alert dimensions: "{"name":"Computer","value":" "}, {"name":"RenderedDescription","value":"Scope ID 10.140.142.0: PercentageInUse is greater than 95%. (0 leases available)"}	Virtual Server	NOC	Complete
T20251030.0651	Fired: Sev2 AM Alert Windows Automatic Services Monitoring Alert (on) at 2025-10-30T20:56:49.0918422Z	Alert dimensions: "{"name":"Computer","value":" "},{"name":"SvcDisplayName","value":"Windows Deployment Services Server"}" View the alert in Azure Monitor: https://portal.azure.com/#view/Mi	Virtual Server	NOC	Complete
T20251029.0236	Fired: Sev2 AM Alert Dell OpenManage Server Administrator (OMSA) Event received (on) at 2025-10-29T15:07:36.2234279Z	Alert dimensions: "{"name":"Computer","value":" "},{"name":"ParameterXml","value":" <Param>Severity: Critical, Category: System Health, MessageID: RDU0012, Message: Power supply	Physical Server	NOC	Complete
T20251029.0037	Fired: Sev2 AM Alert AD DC Directory Services Warning Events (on) at 2025-10-29T08:26:34.7713329Z	Alert dimensions: "{"name":"Computer","value":" "}, {"name":"RenderedDescription","value":"Active Directory Domain Services was unable to establish a connection with the global cata	Virtual Server	NOC	Complete

DEMO

Integrate Azure Monitor alerts from servers with your ITSM system

Artifacts: Entra ID Service Principal, Azure Monitor Alert Rules, Action Groups, Alert processing rules, Logic Apps, and Key Vault



MMS

Project plan to deploy and customize the solution in your environment

1. Create a service principal in the ITSM application and user-assigned managed identity in Azure.
2. Get data from the ITSM needed for API connection.
3. Create an Azure Key vault and populate it with necessary data and secret to connect to the ITSM API.
4. Grant the Azure service principal RBAC permission to read secrets from the Key vault.
5. Create and authorize the Logic app API connection to Key vault.
6. Deploy the two (2) Logic apps from GitHub.
7. Add the outbound IP ranges of the Logic app **Azure-Monitor-Alert-ITSM-HTTP-API** to the Firewall of the Key vault.
8. Customize the Logic apps' parameters to fit your organization.
9. Customize the Logic apps' ticket creation and ticket update "HTTP" task outputs to match expected and desired data fields relevant to your ITSM application.
10. Create an Action group that includes the Logic app **Azure-Monitor-Alert-ITSM-HTTP-API** as an action and associate the Action group with one or more Azure Monitor alert rules.
11. Cause Azure Monitor alerts to fire and observe Logic app behavior.
12. For each of the Log Analytics, Azure Resource Graph, and Metric alert types, continue to customize the text transformation tasks of Logic app **Azure-Monitor-Alert-ITSM-HTTP-API** until ITSM tickets are correctly created and updated in all scenarios.
13. Configure your ITSM to launch the webhook URL of the Logic app **Azure-Monitor-Close-ITSM-HTTP-API** for each Azure Monitor ticket that is marked completed in the ITSM.
14. Enable Secure inputs and outputs for the Key vault tasks in the Logic app **Azure-Monitor-Alert-ITSM-HTTP-API.**

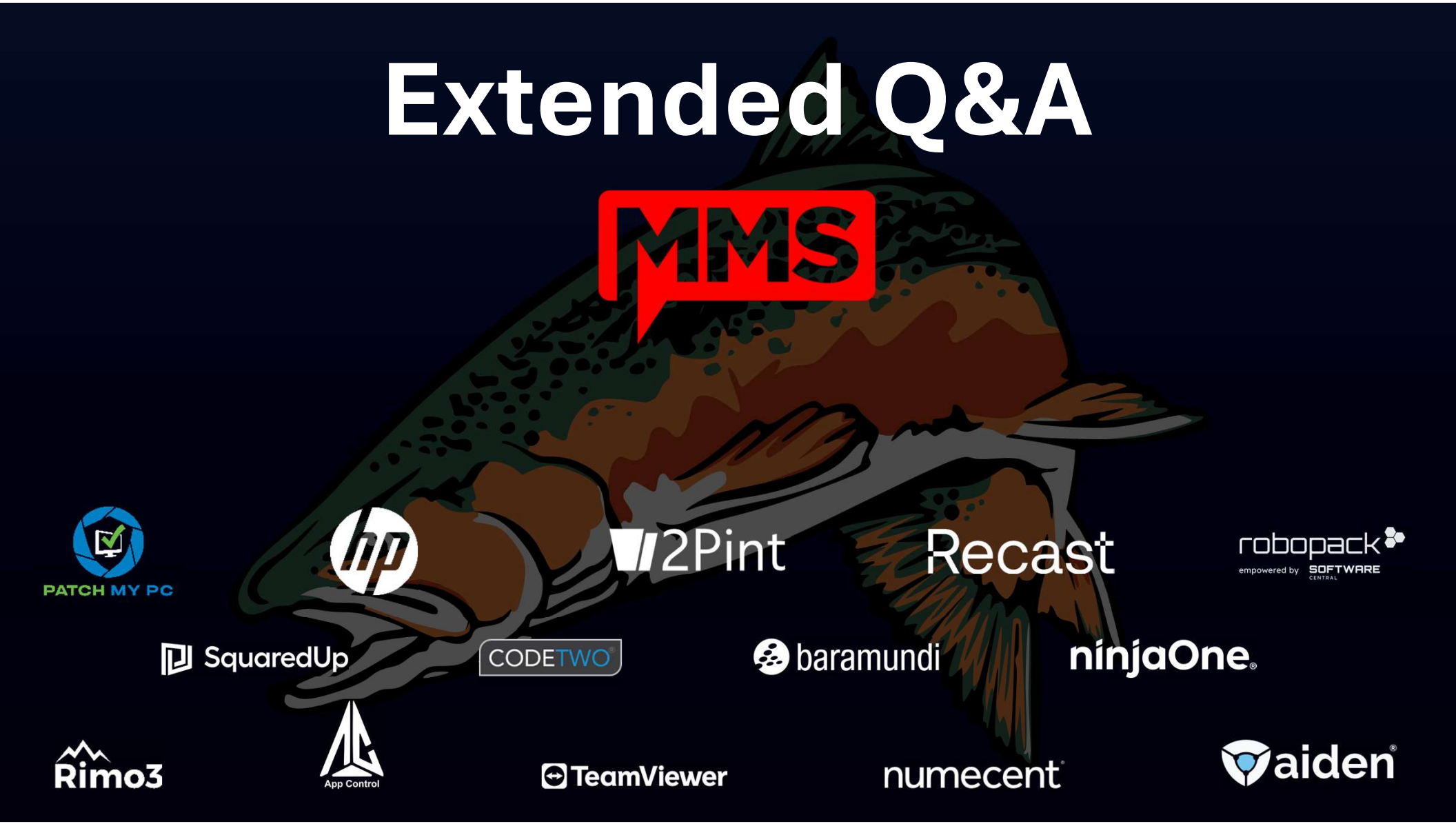
SAVE THE DATES

Oct 25-28, 2026

May 2-6, 2027

Oct 10-13, 2027

Extended Q&A


MMS
PATCH MY PC
hp
2Pint
Recast
robopack
empowered by SOFTWARE CENTRAL
SquaredUp
CODETWO
baramundi
ninjaOne
Rimo3
App Control
TeamViewer
numecent
aiden